

CS2109S: Introduction to AI and Machine Learning

Lecture 8: Intro to Neural Networks

17 March 2026

Announcements

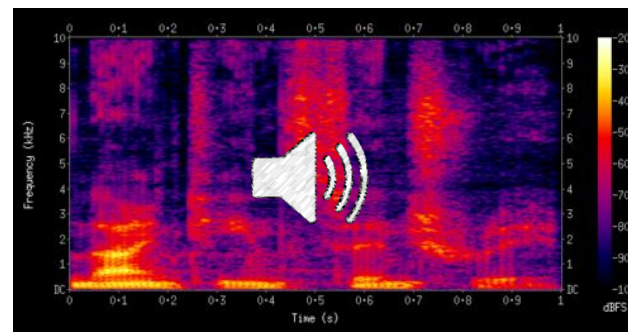
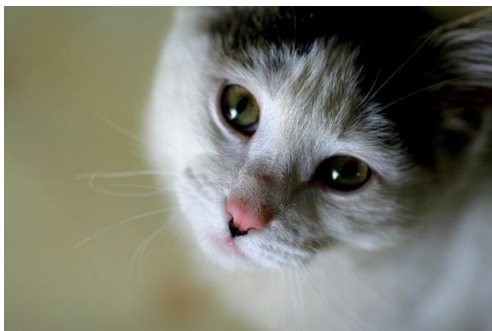
- Midterm
 - Makeup midterm is done.
 - We need to analyze all results.
 - Hopefully, we can release the midterm results by the end of next week.
- Please start working on the Capstone Project.
 - It's due in less than a month!

Outline

- **Neural networks**
 - Neuron model
 - Artificial neural networks
 - Feed-forward neural networks
 - Fully-connected neural networks
- Tasks for feed-forward neural networks
 - Regression: single/multi output
 - Classification: binary, multi-class
- Learning via gradient descent
 - Backpropagation

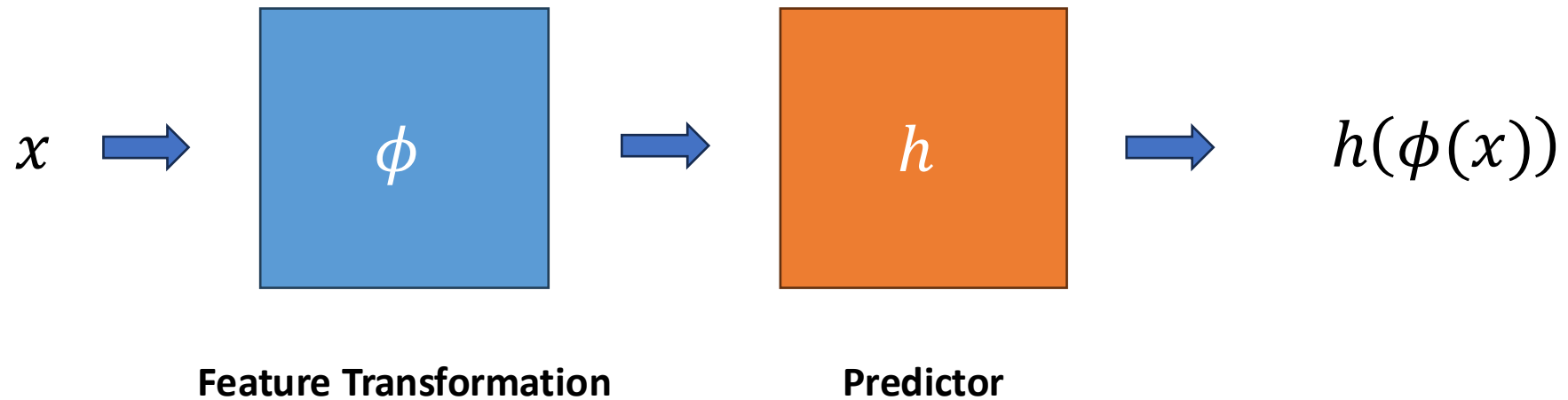
Introduction

- So far, we deal with problems with “relatively simple” ground truth.
- When dealing with non-linear problems, we “guess” the relationships between input and output to decide the feature transformations.
- What about problems with the following input?



- We can handcraft useful feature transformations, however
 - Maybe difficult to do.
 - May not be good.

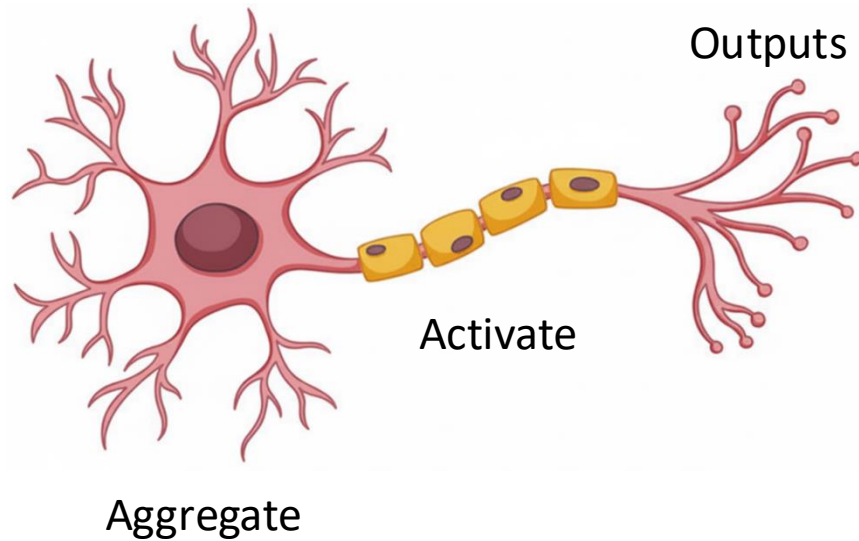
Learned Feature Transformation



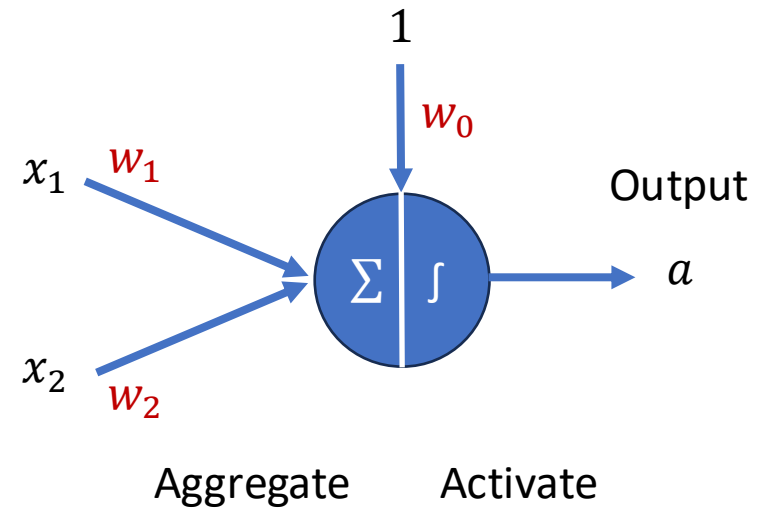
- Instead of handcrafting features, **learn** a feature transformation function ϕ .
- Jointly learn ϕ and h using training data with the usual loss function.
- Example: **neural networks**

Neuron

Neuron in brain:



Neuron model:



Usually (but not always):

- Aggregation function: sum.
- Activation function: non-linear function.

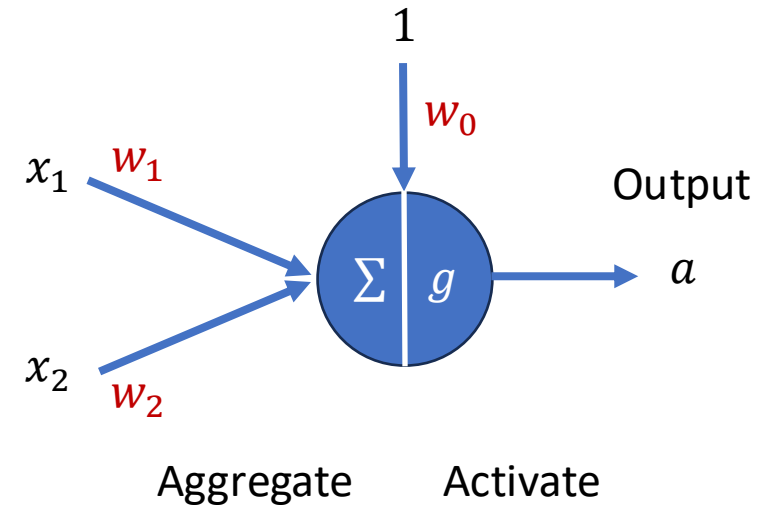
Neuron Model

A neuron model computes a weighted sum of input features and passes the result through an **activation function**.

$$z = \sum_{j=0}^d w_j x_j$$

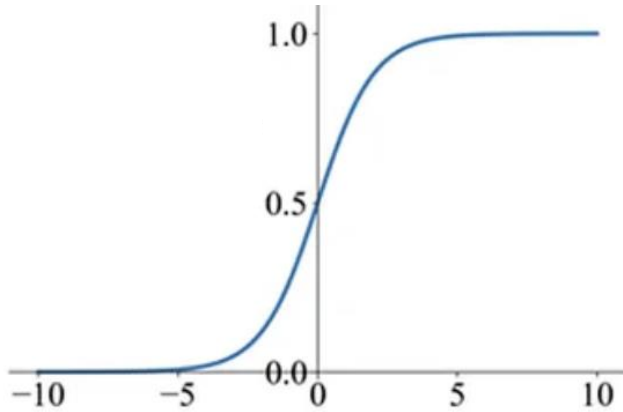
$$a = g(z)$$

Here, $w_0 \dots w_d$ are weights and $x_0 \dots x_d$ are input features, where d is the dimension of the input features. g represents the activation function.

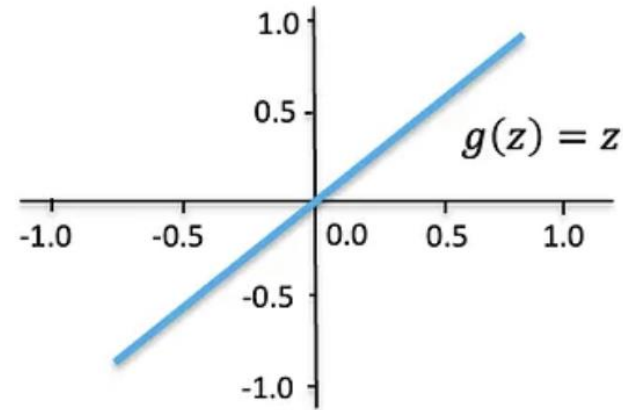


- Linear regression model: neuron model with identity activation, i.e., $g(z) = z$.
- Logistic regression model: neuron model with sigmoid activation, i.e., $g = \sigma$.
- In both of these models, the output $\hat{y} = a$

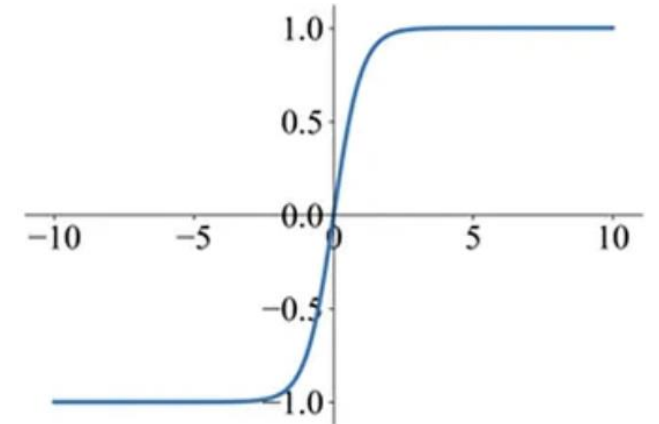
Common Activation Functions



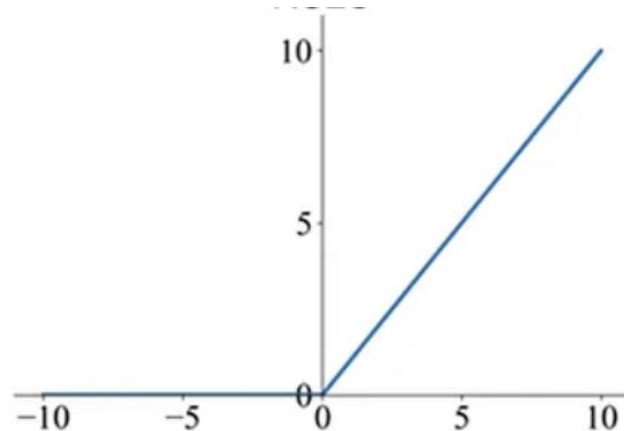
Sigmoid: $g(z) = \sigma(z) = \frac{1}{1+e^{-z}}$



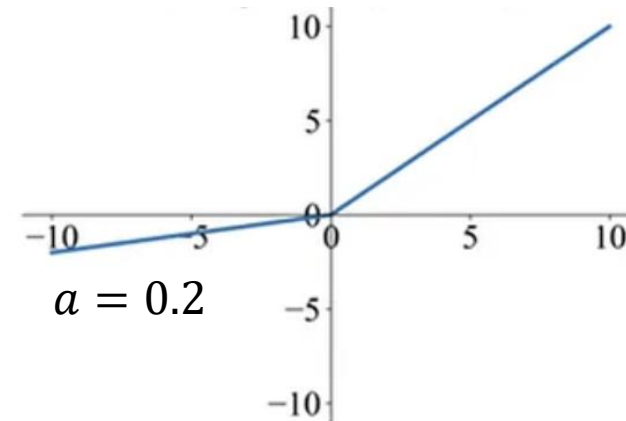
Identity: $g(z) = z$



Tanh: $g(z) = \tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$



ReLU: $g(z) = \max(0, z)$



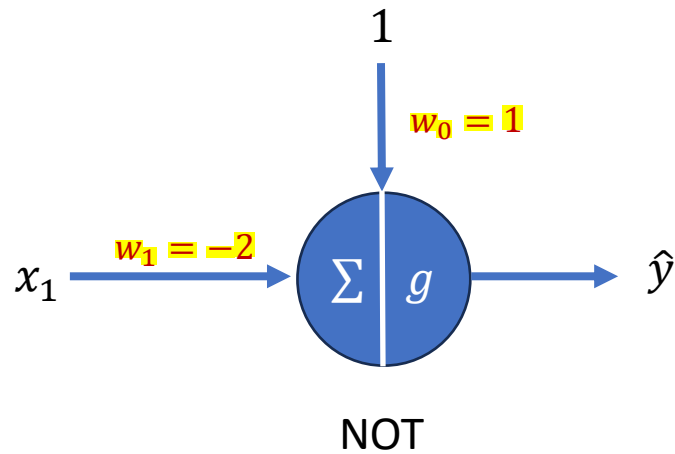
Leaky ReLU: $g(z) = \max(az, z)$

What can we do with a Neuron Model?

It can do what linear/logistic regression model can do.

Example (logic circuit):

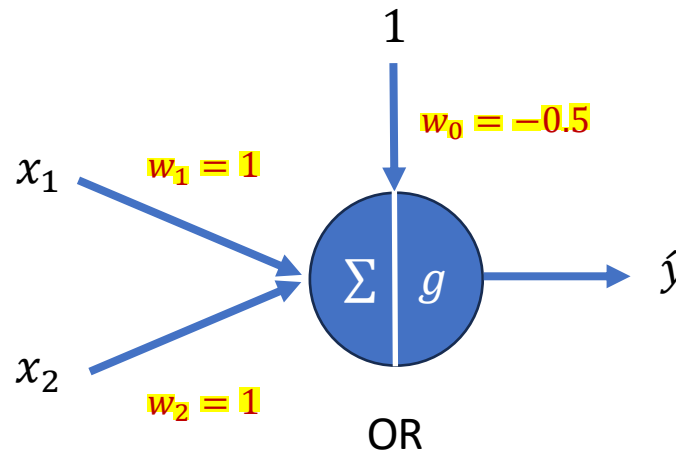
Let g be sigmoid function. Decision threshold = 0.5.



Added description:

Take Boolean inputs $x_1 \in \{0,1\}$. Evaluate model/threshold:

$x_1 = 0 \rightarrow \sigma(-2 * 0 + 1) = \sigma(1) > 0.5 \rightarrow 1$
 $x_1 = 1 \rightarrow \sigma(-2 * 1 + 1) = \sigma(-1) < 0.5 \rightarrow 0$

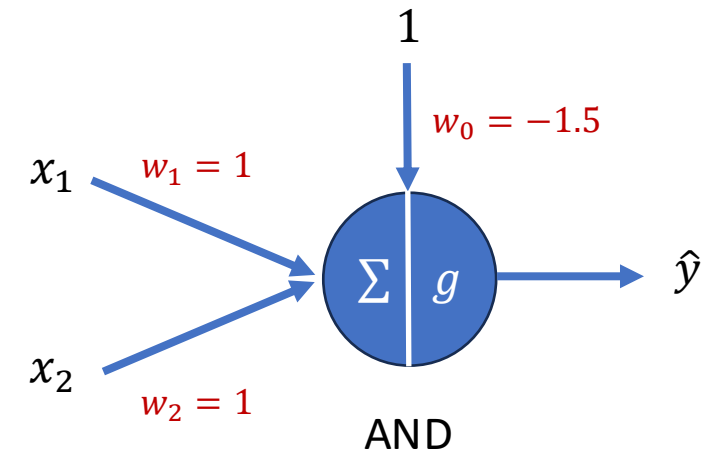


Added description:

Take Boolean inputs $x_1, x_2 \in \{0,1\}$. Evaluate model/threshold:

$00 \rightarrow \sigma(1 * 0 + 1 * 0 - 0.5) = \sigma(-0.5) < 0.5 \rightarrow 0$
 $01 \rightarrow \sigma(1 * 0 + 1 * 1 - 0.5) = \sigma(0.5) > 0.5 \rightarrow 1$
 $10 \rightarrow \sigma(1 * 1 + 1 * 0 - 0.5) = \sigma(0.5) > 0.5 \rightarrow 1$
 $11 \rightarrow \sigma(1 * 1 + 1 * 1 - 0.5) = \sigma(1.5) > 0.5 \rightarrow 1$

Note that different model weights can work as well.

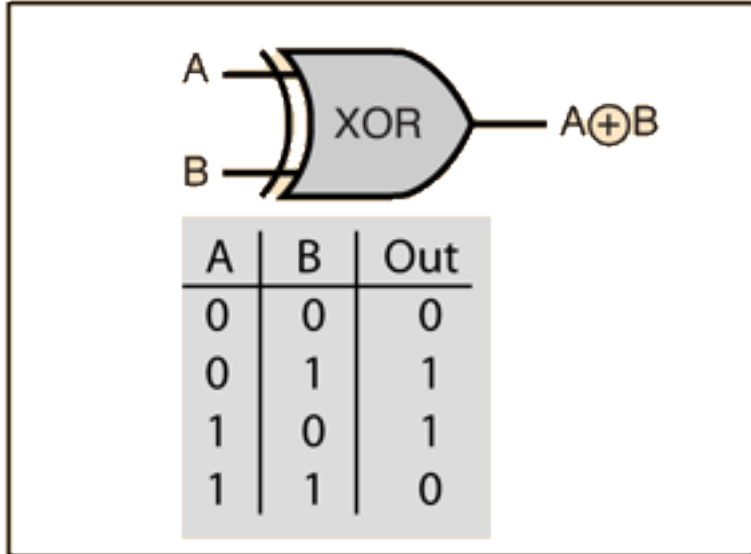


Added description:

Take Boolean inputs $x_1, x_2 \in \{0,1\}$. Evaluate model/threshold:

$00 \rightarrow \sigma(1 * 0 + 1 * 0 - 1.5) = \sigma(-1.5) < 0.5 \rightarrow 0$
 $01 \rightarrow \sigma(1 * 0 + 1 * 1 - 1.5) = \sigma(-0.5) < 0.5 \rightarrow 0$
 $10 \rightarrow \sigma(1 * 1 + 1 * 0 - 1.5) = \sigma(-0.5) < 0.5 \rightarrow 0$
 $11 \rightarrow \sigma(1 * 1 + 1 * 1 - 1.5) = \sigma(0.5) > 0.5 \rightarrow 1$

What about XOR?



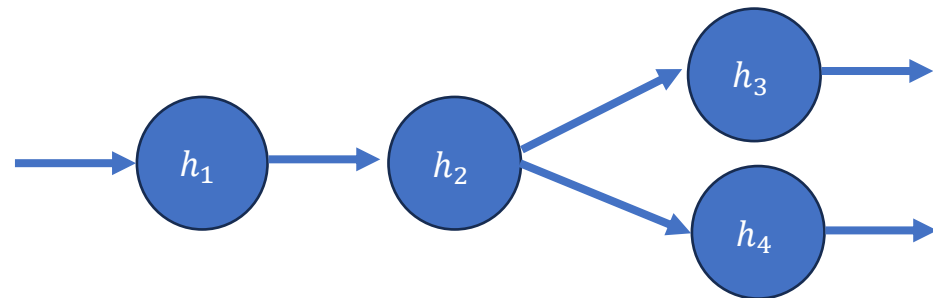
1. $A \oplus B = A\bar{B} + B\bar{A}$

(Artificial) Neural Networks

A neural network is (traditionally) a network of neuron models.

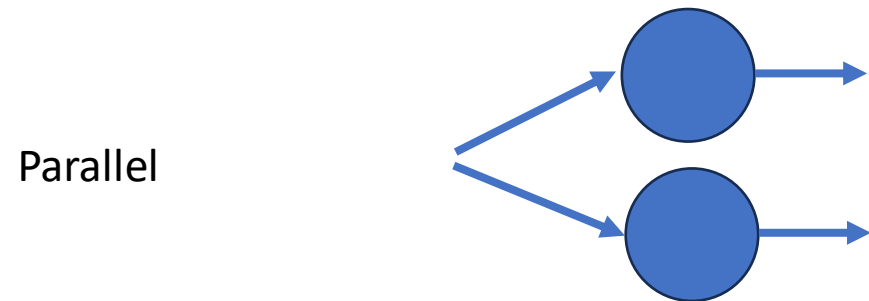
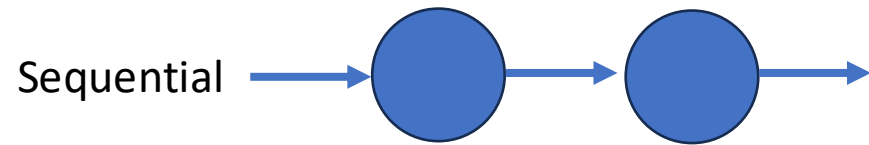
Formally, a neural network is a computation graph $G = (V, E)$.

- Node $v \in V$: contains function h_v parameterized by $w^{[v]}$.
 - Traditionally, a neuron model, in general can be any function (week 9+).
- Edge $e \in E$: represent data flow.
- Parameters $w = \bigcup_v w^{[v]}$.

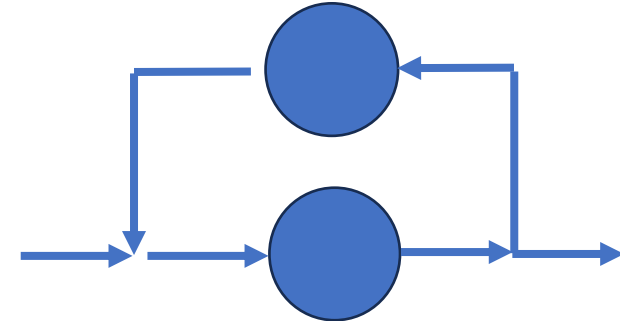
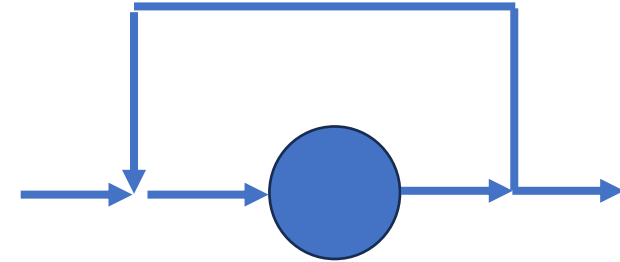


$$w = w^{[1]} \cup w^{[2]} \cup w^{[3]} \cup w^{[4]}$$

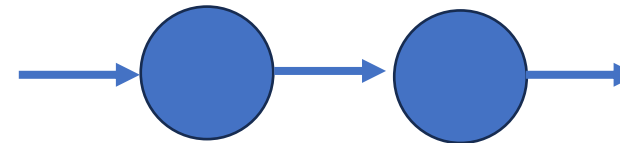
Neural Networks: Examples



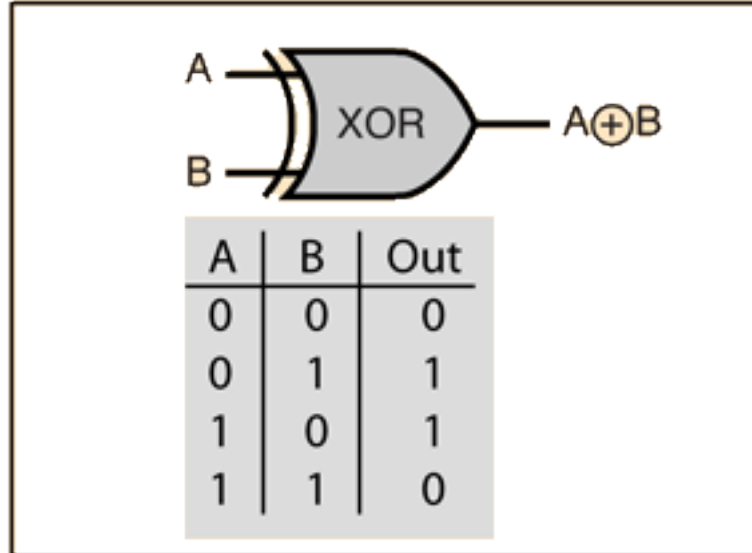
Cycles



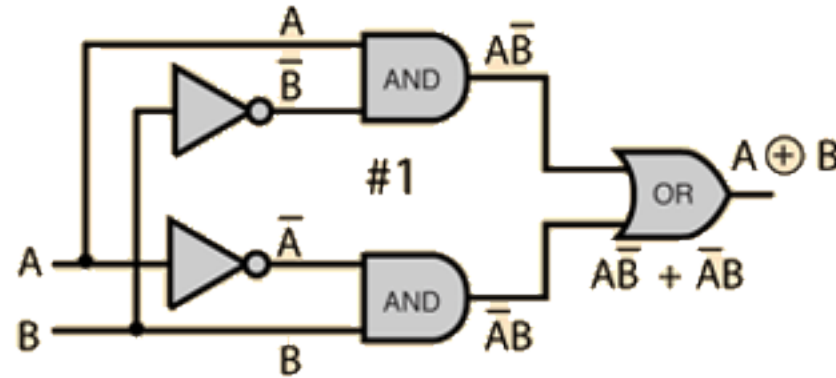
No cycles



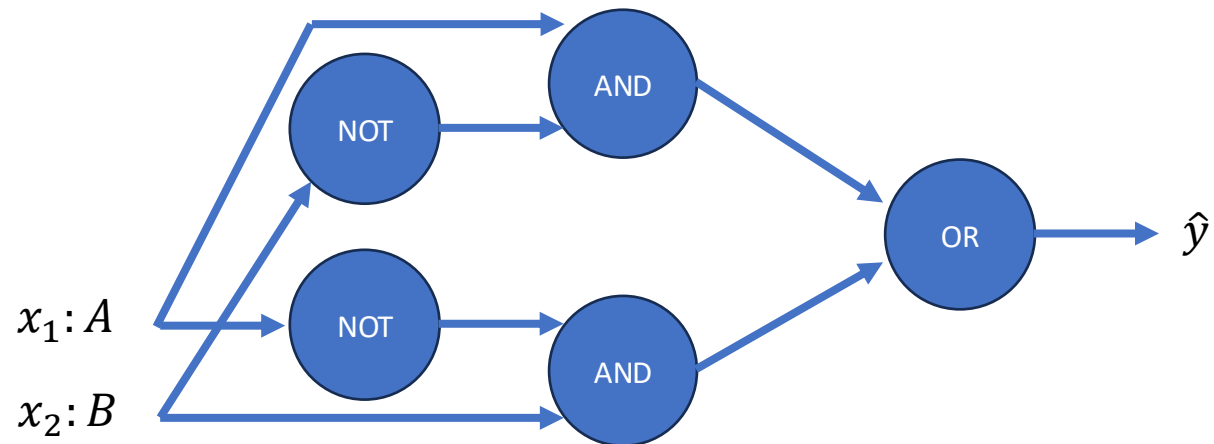
Neural Networks: XOR



1. $A \oplus B = A\bar{B} + \bar{A}B$

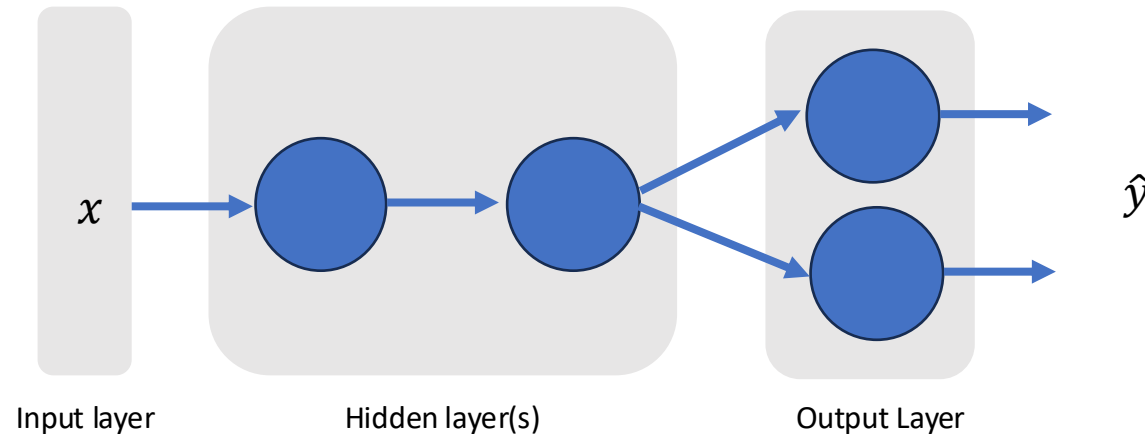


Using neuron models defined before, define neural networks:



Feed-forward Neural Networks (FFNNs)

A special case of neural networks where the computation graph has designated input and output and there are no cycles. i.e., the computation graph is a directed acyclic graph (DAG).



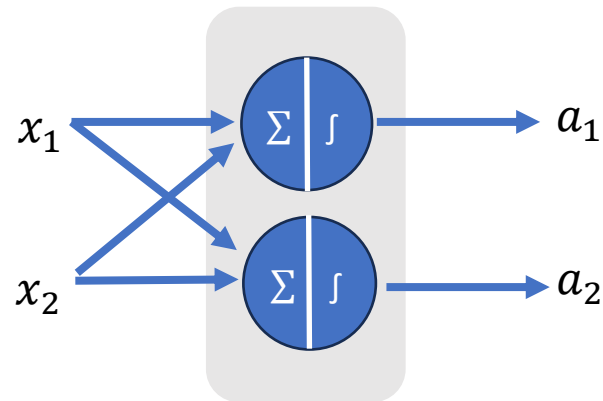
Definition: A layer in a neural network is a parameterized map $l_w: X_{in} \rightarrow X_{out}$ that takes an input from a domain X_{in} (e.g., vector) and produces an output in a domain X_{out} (e.g., an intermediate or final output vector).

- Refer to a group of nodes that are applied together (often in parallel) as one unit.

Deep learning: neural networks with more than one hidden layer.

Fully-connected (FC) Layer

A layer that is composed of parallel neuron models.

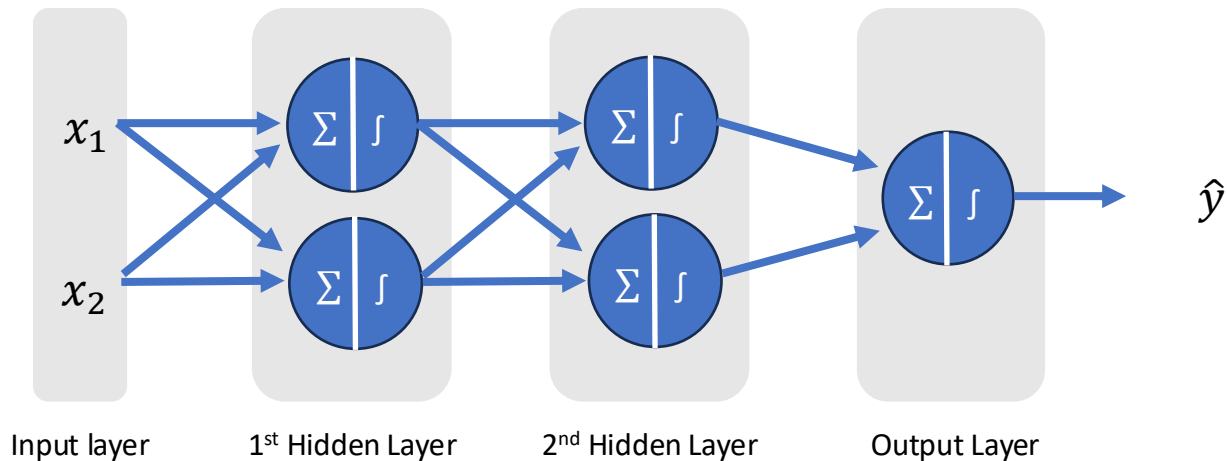


Fully-connected layer

Fully connected: all input nodes are connected to all neuron models.

Fully-connected Neural Networks (FCNNs)

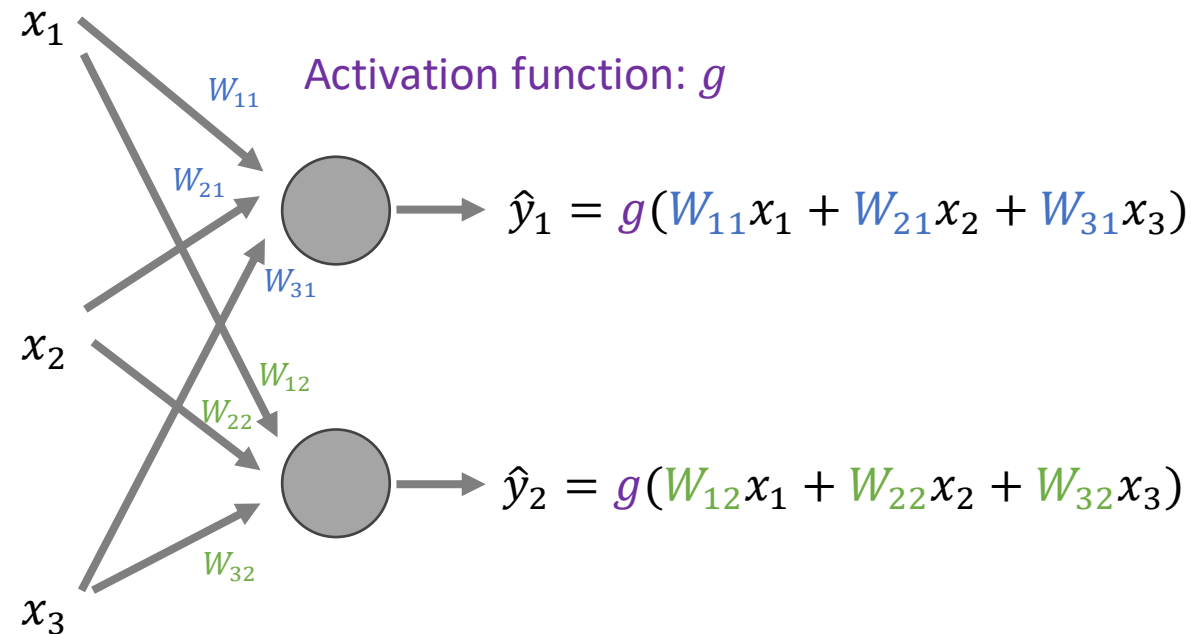
A special case of feed-forward neural networks that are composed of sequential fully-connected layers.



- **Layer:** a group of nodes that are applied in parallel.
- **Depth:** the number of layers applied sequentially.
 - Our convention: **Depth** does not count the input layer.
- **Width:** the number of neuron models in a layer.

FCNN and Matrix Multiplication

1-layer (No bias)



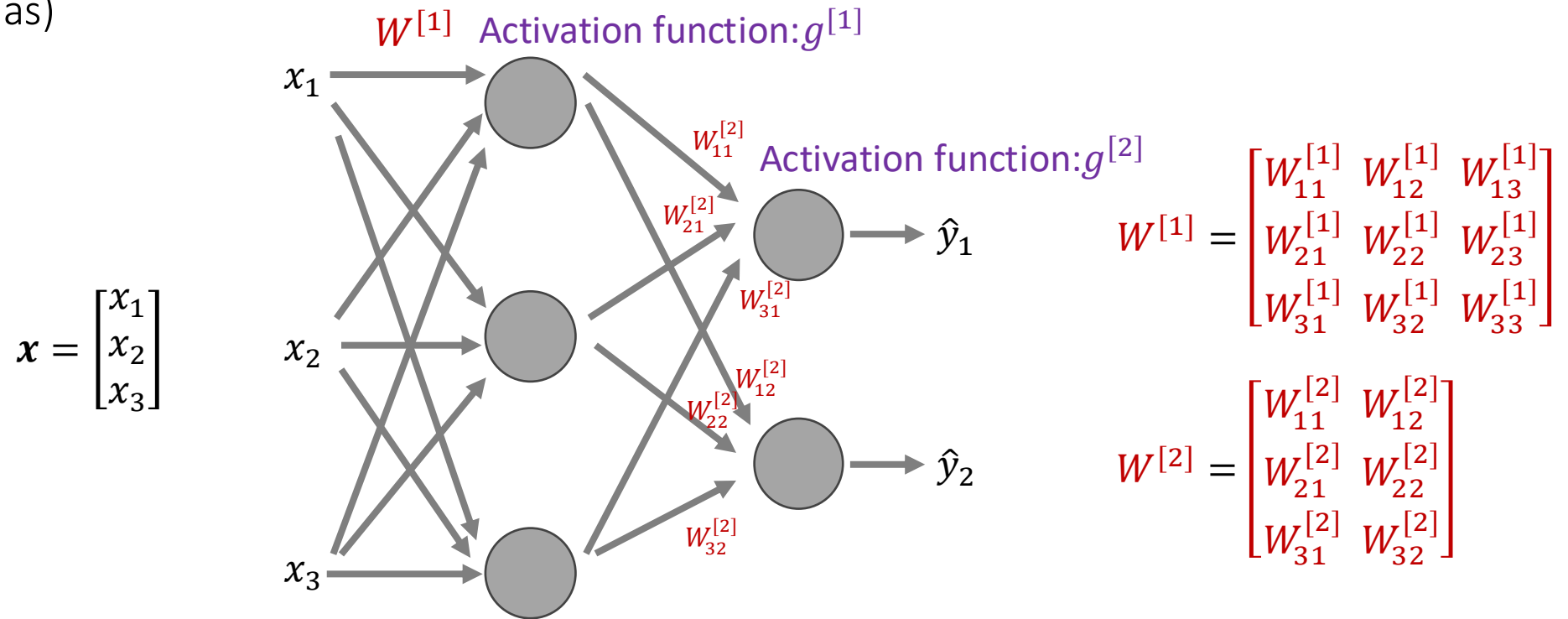
Input (number of weights per neuron / input variables)

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} \quad \mathbf{W} = \begin{bmatrix} W_{11} & W_{12} \\ W_{21} & W_{22} \\ W_{31} & W_{32} \end{bmatrix} \quad \hat{\mathbf{y}} = g(\mathbf{W}^T \mathbf{x}) = g\left(\begin{bmatrix} W_{11} & W_{12} \\ W_{21} & W_{22} \\ W_{31} & W_{32} \end{bmatrix}^T \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}\right) = g\left(\begin{bmatrix} W_{11} & W_{21} & W_{31} \\ W_{12} & W_{22} & W_{32} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}\right) = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \end{bmatrix}$$

Output (number of layer's neurons / output variables)

FCNN and Matrix Multiplication

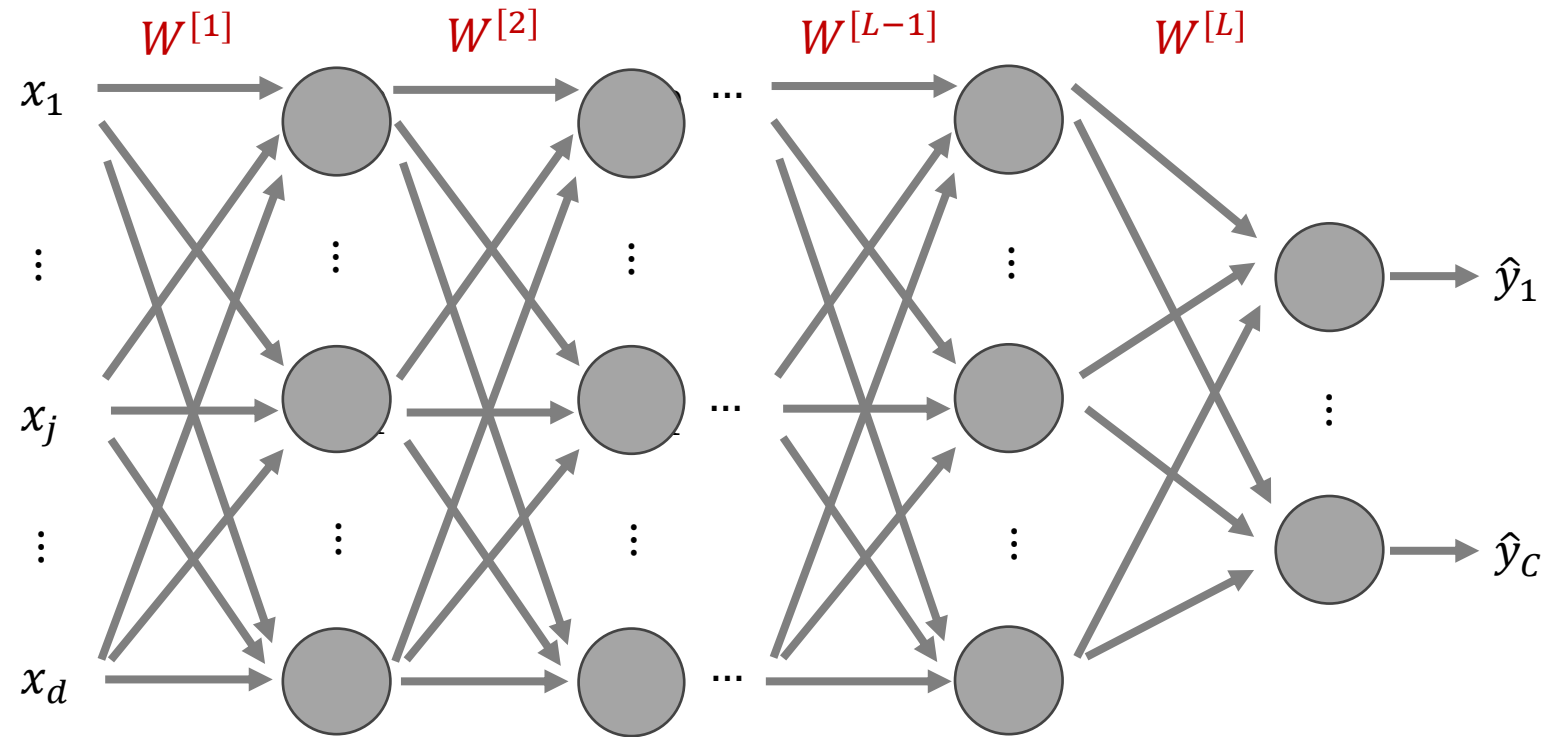
2-layer (No bias)



$$\hat{\mathbf{y}} = g^{[2]} \left(W^{[2]T} g^{[1]} \left(W^{[1]T} \mathbf{x} \right) \right) = g^{[2]} \left(\begin{bmatrix} W_{11}^{[2]} & W_{12}^{[2]} \\ W_{21}^{[2]} & W_{22}^{[2]} \\ W_{31}^{[2]} & W_{32}^{[2]} \end{bmatrix}^T g^{[1]} \left(\begin{bmatrix} W_{11}^{[1]} & W_{12}^{[1]} & W_{13}^{[1]} \\ W_{21}^{[1]} & W_{22}^{[1]} & W_{23}^{[1]} \\ W_{31}^{[1]} & W_{32}^{[1]} & W_{33}^{[1]} \end{bmatrix}^T \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} \right) \right) = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \end{bmatrix}$$

FCNN and Matrix Multiplication

L-layers (No bias)



$$\hat{\mathbf{y}} = g^{[L]} \left(W^{[L]T} g^{[L-1]} \left(W^{[L-1]T} g^{[L]} \left(W^{[L]T} \dots g^{[2]} \left(W^{[2]T} g^{[1]} \left(W^{[1]T} \mathbf{x} \right) \right) \right) \right) \right) = \begin{bmatrix} \hat{y}_1 \\ \dots \\ \hat{y}_c \end{bmatrix}$$

FCNN: Analysis

Theorem (universal function approximation, informal):

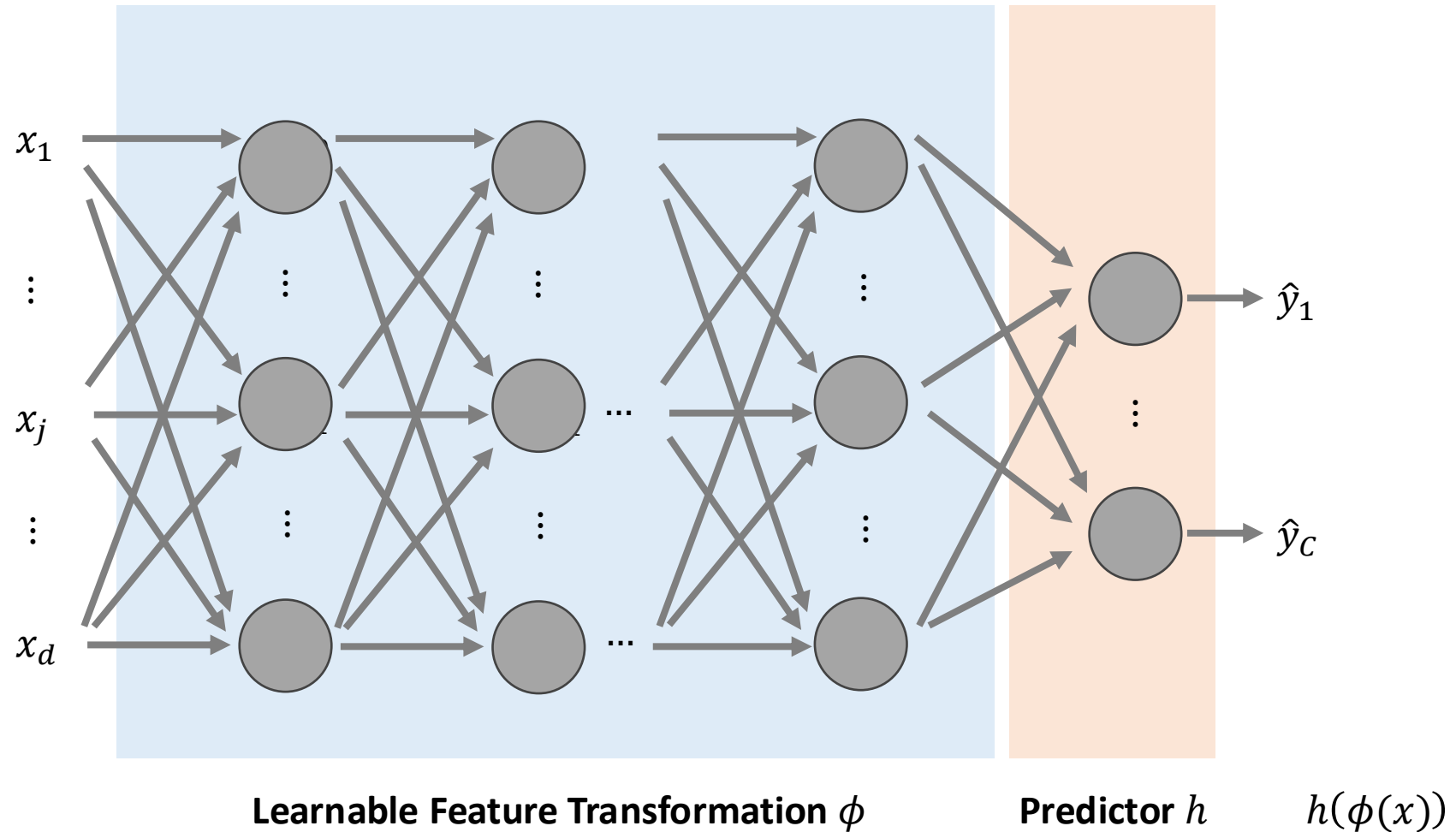
A single hidden layer fully connected neural networks with sufficient width and non-linear activation function (see below) can approximate any continuous function within a bounded region of multivariate input.

- Cybenko et al. (1989): Sigmoid
- Hornik et al. (1991): Very broad class of activation functions
- Leshno et al. (1993): Some class of non-polynomial functions
- Leshno et al. (1993), Lu et al (2017), Hannin & Selke (2018): ReLU

Why do we even bother with networks with multiple hidden layers?

(Feed-forward) General View of ML

With feed-forward neural networks



A mostly complete chart of Neural Networks

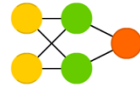
©2019 Fjodor van Veen & Stefan Leijnen asimovinstitute.org

-  Input Cell
-  Backfed Input Cell
-  Noisy Input Cell
-  Hidden Cell
-  Probabilistic Hidden Cell
-  Spiking Hidden Cell
-  Capsule Cell
-  Output Cell
-  Match Input Output Cell
-  Recurrent Cell
-  Memory Cell
-  Gated Memory Cell
-  Kernel
-  Convolution or Pool

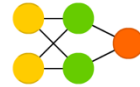
Perceptron (P)



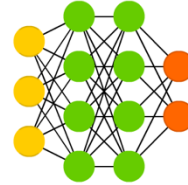
Feed Forward (FF)



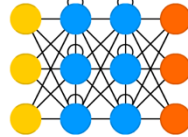
Radial Basis Network (RBF)



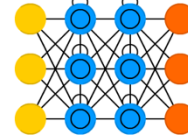
Deep Feed Forward (DFF)



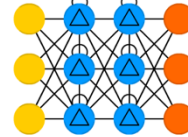
Recurrent Neural Network (RNN)



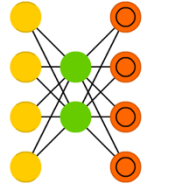
Long / Short Term Memory (LSTM)



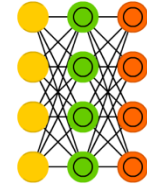
Gated Recurrent Unit (GRU)



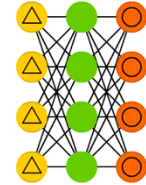
Auto Encoder (AE)



Variational AE (VAE)



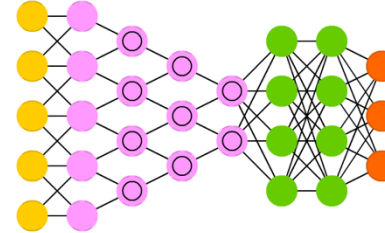
Denoising AE (DAE)



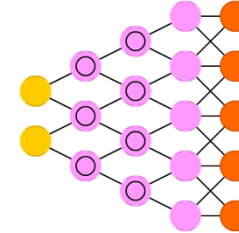
Sparse AE (SAE)



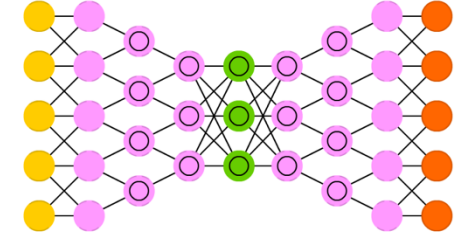
Deep Convolutional Network (DCN)



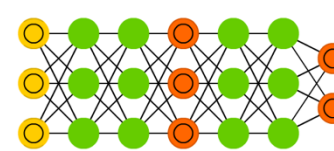
Deconvolutional Network (DN)



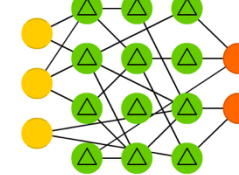
Deep Convolutional Inverse Graphics Network (DCIGN)



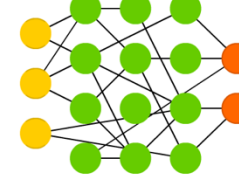
Generative Adversarial Network (GAN)



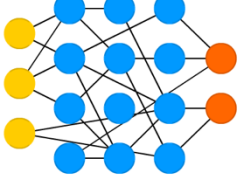
Liquid State Machine (LSM)



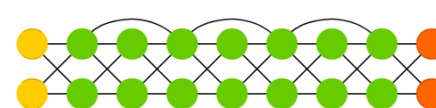
Extreme Learning Machine (ELM)



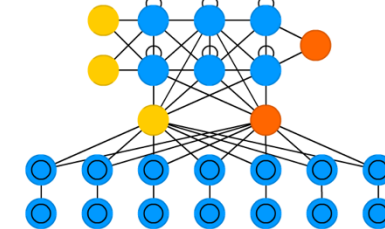
Echo State Network (ESN)



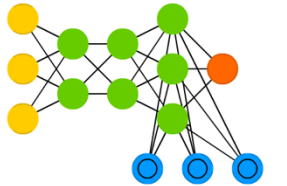
Deep Residual Network (DRN)



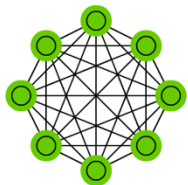
Differentiable Neural Computer (DNC)



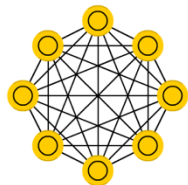
Neural Turing Machine (NTM)



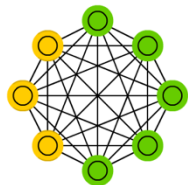
Markov Chain (MC)



Hopfield Network (HN)



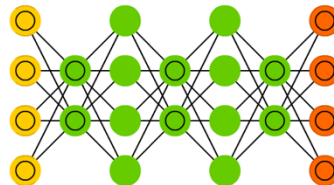
Boltzmann Machine (BM)



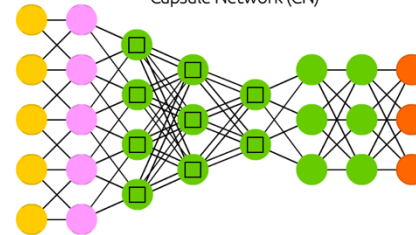
Restricted BM (RBM)



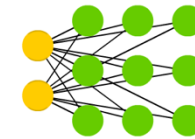
Deep Belief Network (DBN)



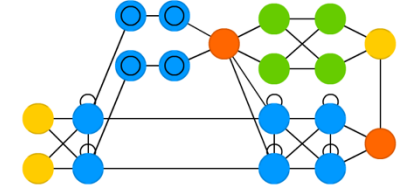
Capsule Network (CN)



Kohonen Network (KN)

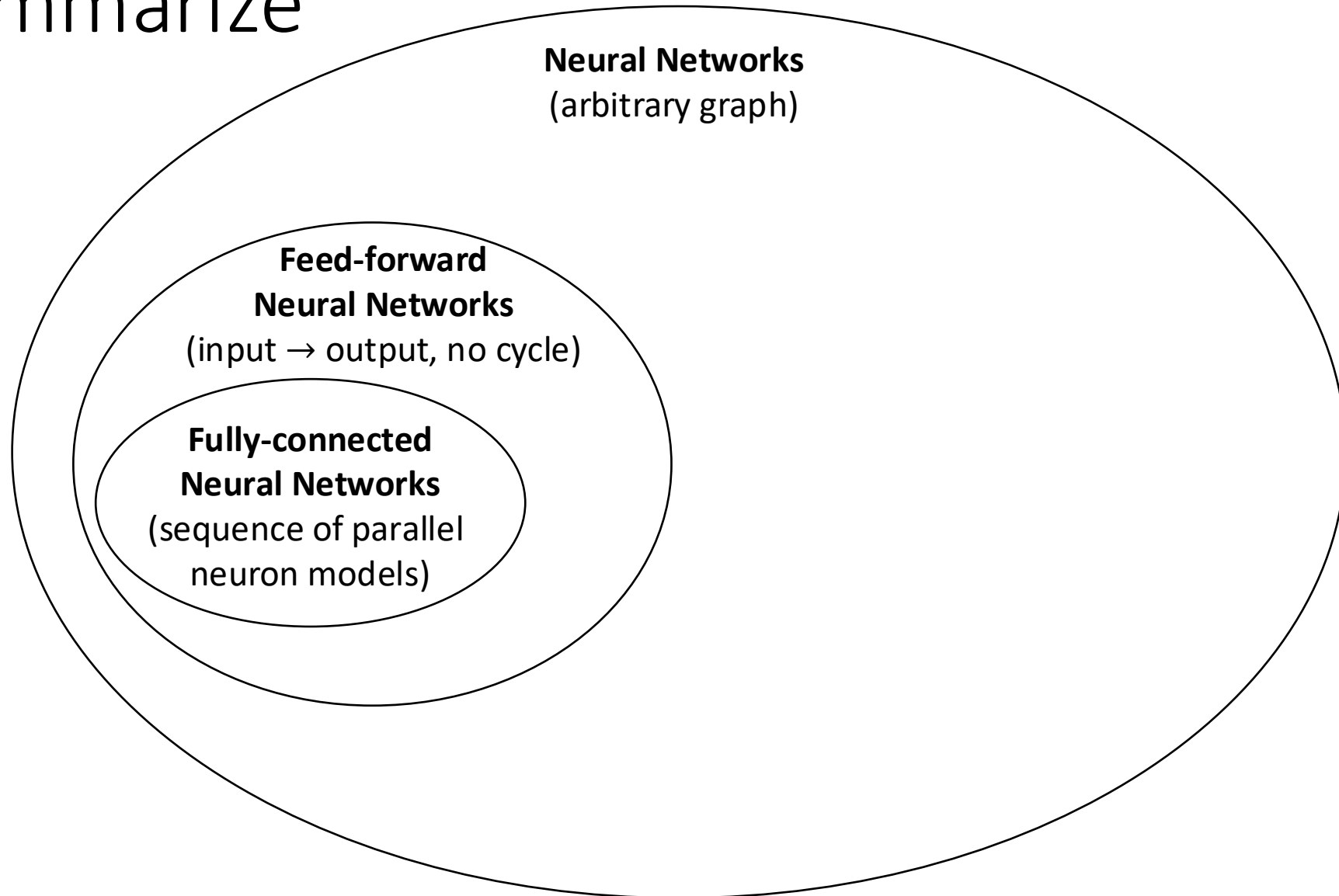


Attention Network (AN)



Source: <https://www.asimovinstitute.org/neural-network-zoo/>

To summarize



Break

UNCOVERING THE BRAIN'S BIGGEST SECRET

TEDEd

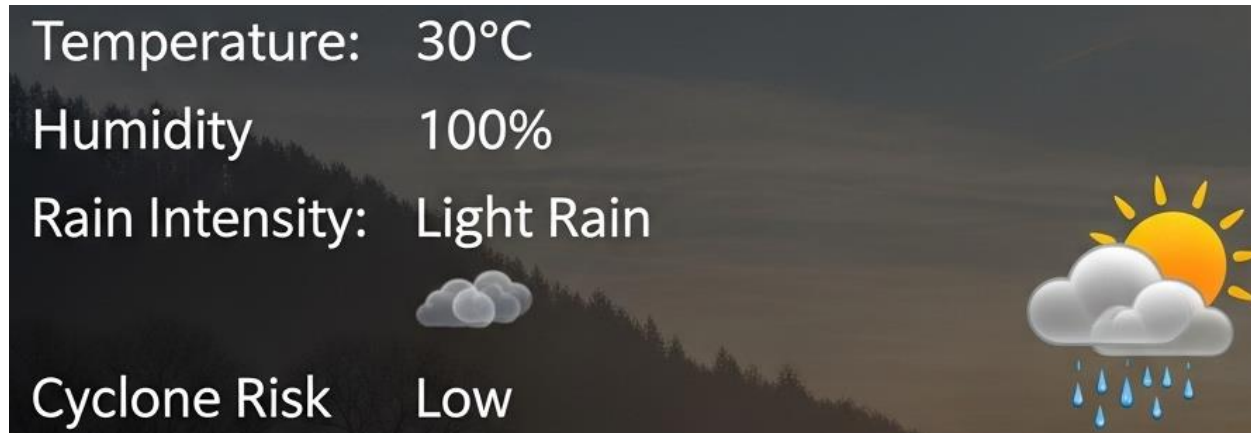


Outline

- Neural networks
 - Neuron model
 - Artificial neural networks
 - Feed-forward neural networks
 - Fully-connected neural networks
- **Tasks for feed-forward neural networks**
 - Regression: single/multi output
 - Classification: binary, multi-class
- Learning via gradient descent
 - Backpropagation

Example: Weather Forecast

- **Input:** temperature, humidity, pressure, wind speed
- **Tasks:** Predict next day:
 - Temperature, humidity, etc (regression)
 - Rain intensity, cyclone risk, etc (classification)



Data

Suppose:

- We are given n data points.
- Each data point consists of **features** and a **target** variable.
- Each data point has d features and they are described by **real numbers**.
- The target is **task-dependent**.

Data – Math

Suppose

$$D = \{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(\textcolor{red}{n})}, y^{(\textcolor{red}{n})})\},$$

where for all $i \in \{1, \dots, \textcolor{red}{n}\}$

Features: $x^{(i)} \in \mathbb{R}^{\textcolor{red}{d}}$

Targets: $y^{(i)}$ (task-dependent)

Regression: Temperature

Given data point $x \in \mathbb{R}^d$, predict temperature, i.e., $y \in \mathbb{R}$.

Regression problem.

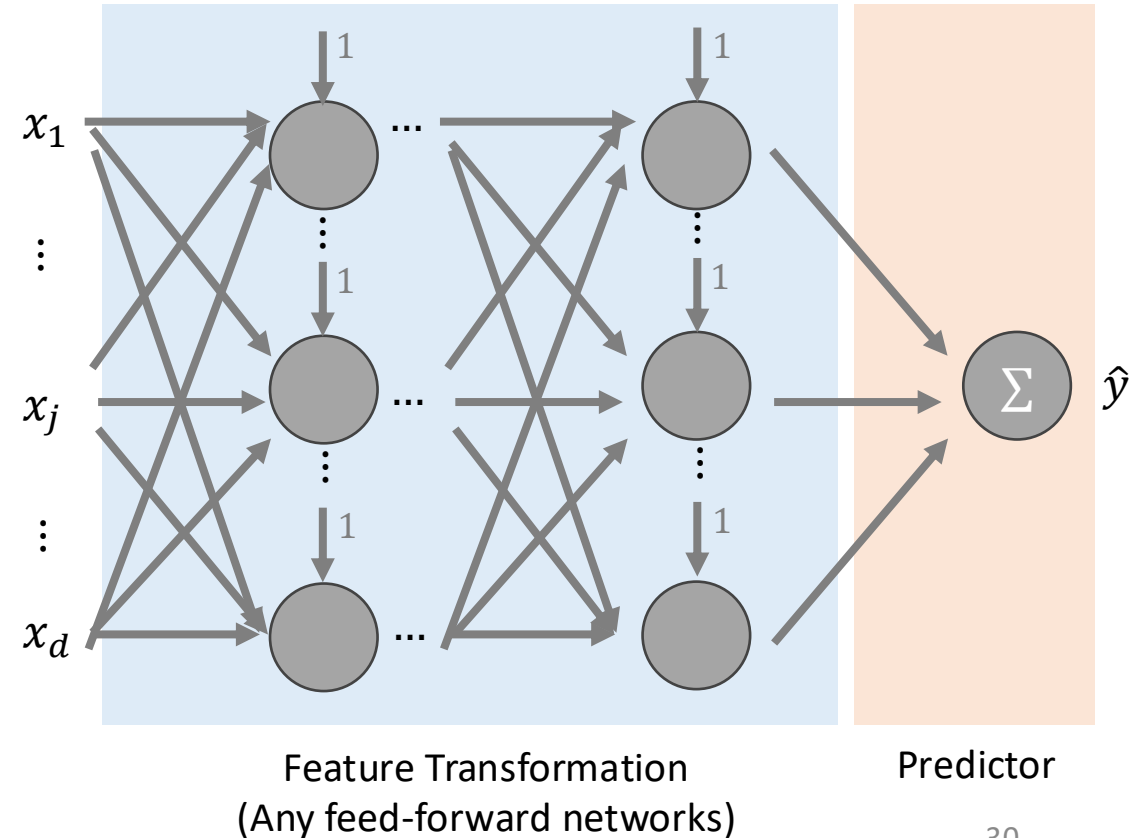
Predictor:

A single neuron with identity activation

Loss function:

$$J_{MSE}(\mathbf{w}) = \frac{1}{2n} \sum_{i=1}^n (\hat{y}^{(i)} - y^{(i)})^2$$

y
30
32.5
10



Regression: Temperature and Humidity

y_1	y_2
30	10
32.5	100
10	85

Given data point $x \in \mathbb{R}^d$, predict temperature and humidity, i.e., $y \in \mathbb{R}^m$, $m = 2$.

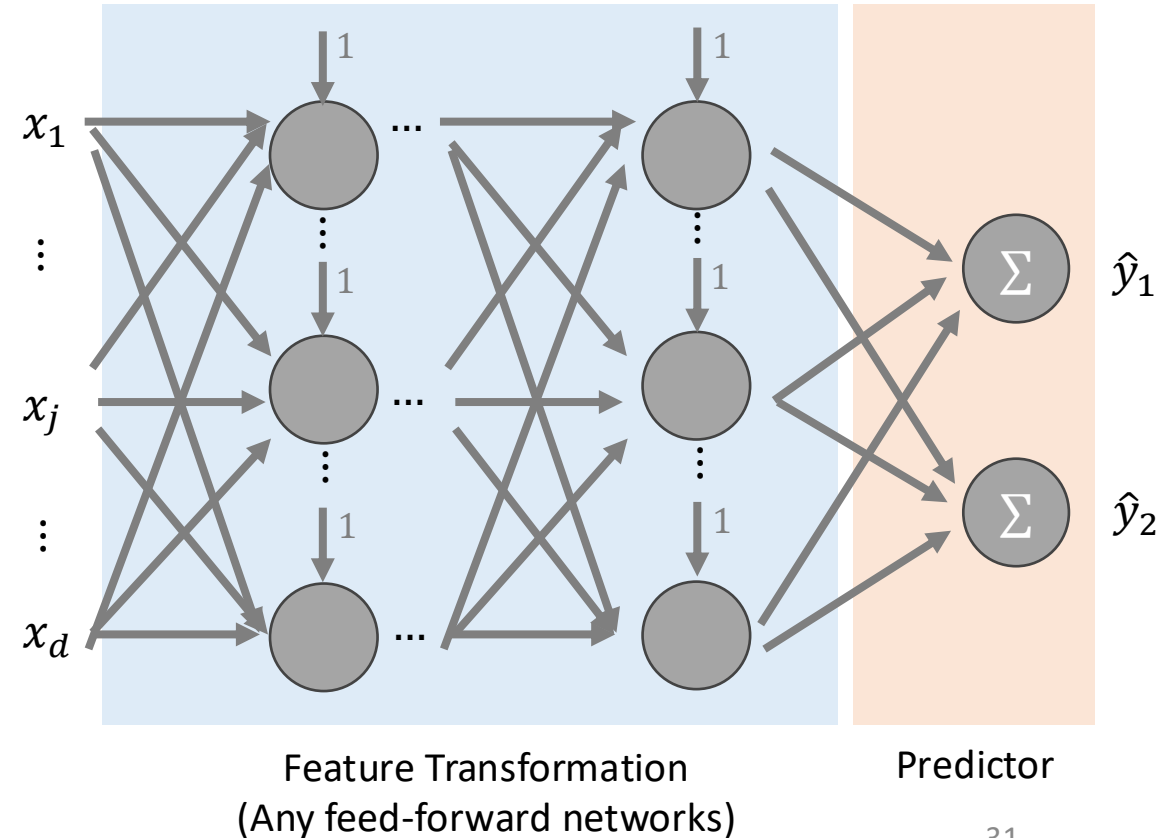
Multi-regression problem.

Predictor:

2 neurons with identity activation

Loss function:

$$J_{MSE}(w) = \frac{1}{2n} \sum_{i=1}^n \sum_{j=1}^m \left(\hat{y}_j^{(i)} - y_j^{(i)} \right)^2$$



y
1
0
1

Classification: Rain or Not

Given data point $x \in \mathbb{R}^d$, predict whether it's going to rain or not, i.e., $y \in \{0,1\}$.

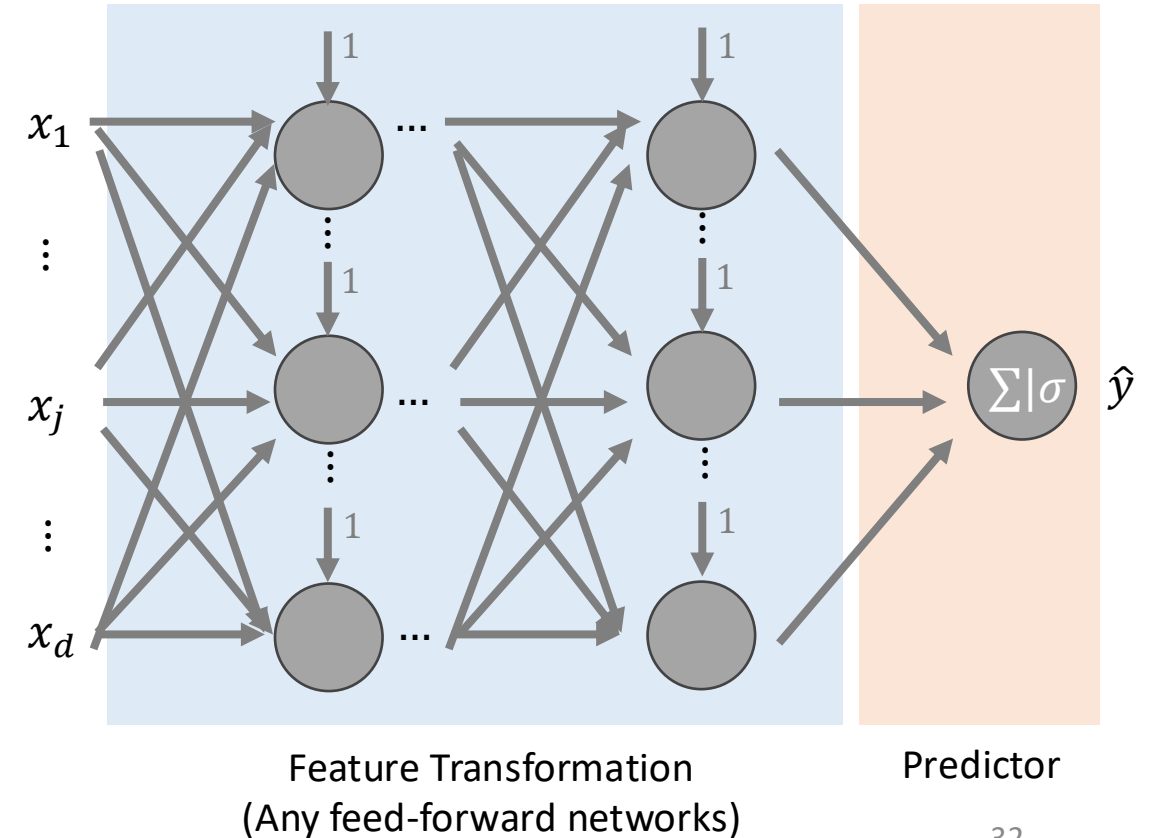
Binary classification problem.

Predictor:

A single neuron with sigmoid activation

Loss function:

$$J_{BCE}(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n BCE(y^{(i)}, \hat{y}^{(i)})$$



Classification: Rain Intensity

Given data point $x \in \mathbb{R}^d$, predict rain intensity (drizzle, light, heavy).

Multi-class classification problem.

y

drizzle

light

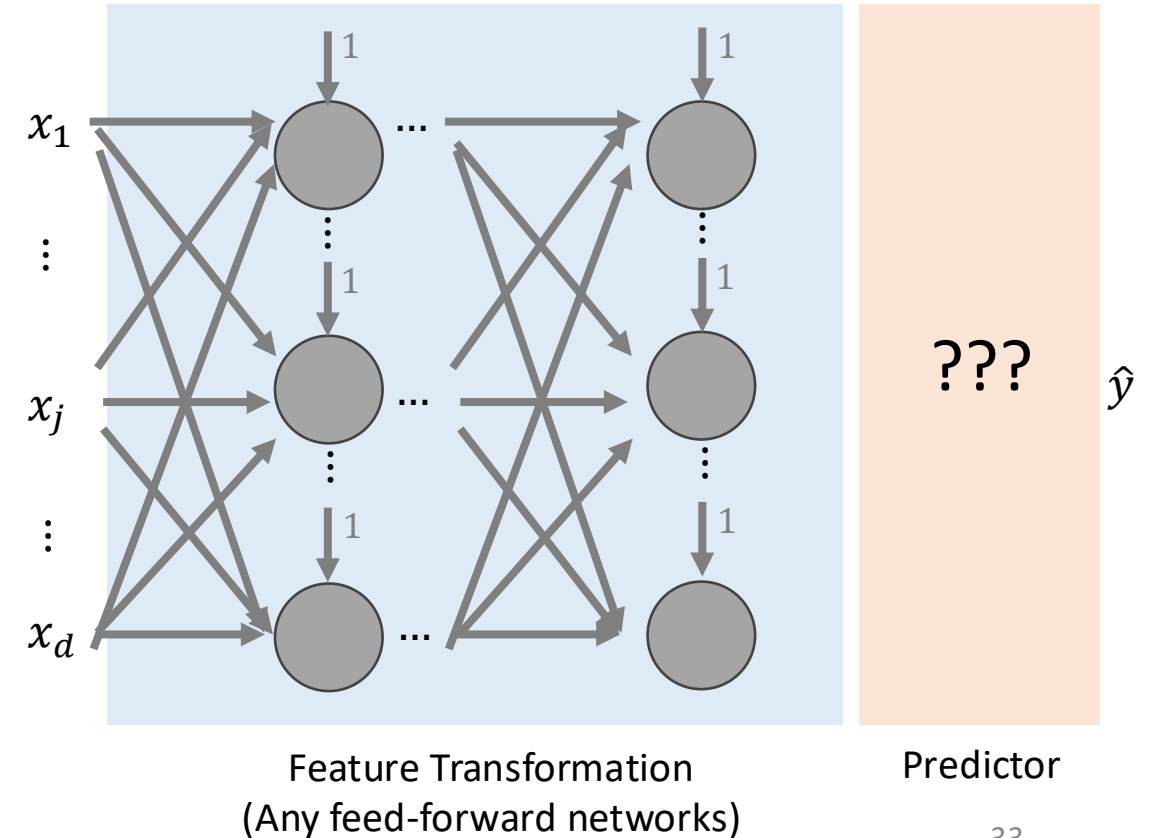
heavy

Predictor:

???

Loss function:

???



Background: Categorical probability

Given a discrete random variable $X \in \{1, 2, \dots, K\}$.

The probability $p(X)$ for the outcomes of this random variable is called a **categorical probability distribution**.

Example: Uniform 6-sided dice.

$$p([1, 2, 3, 4, 5, 6]) = \left[\frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6}, \frac{1}{6} \right]$$



One-hot Encoding

One-hot encoding is a way to represent a categorical label as a categorical probability distribution vector.

For K number of classes, one-hot vector $\in \{0,1\}^K$.

Each class is represented by a vector that has:

- 1 in the position corresponding to that class, and
- 0s everywhere else

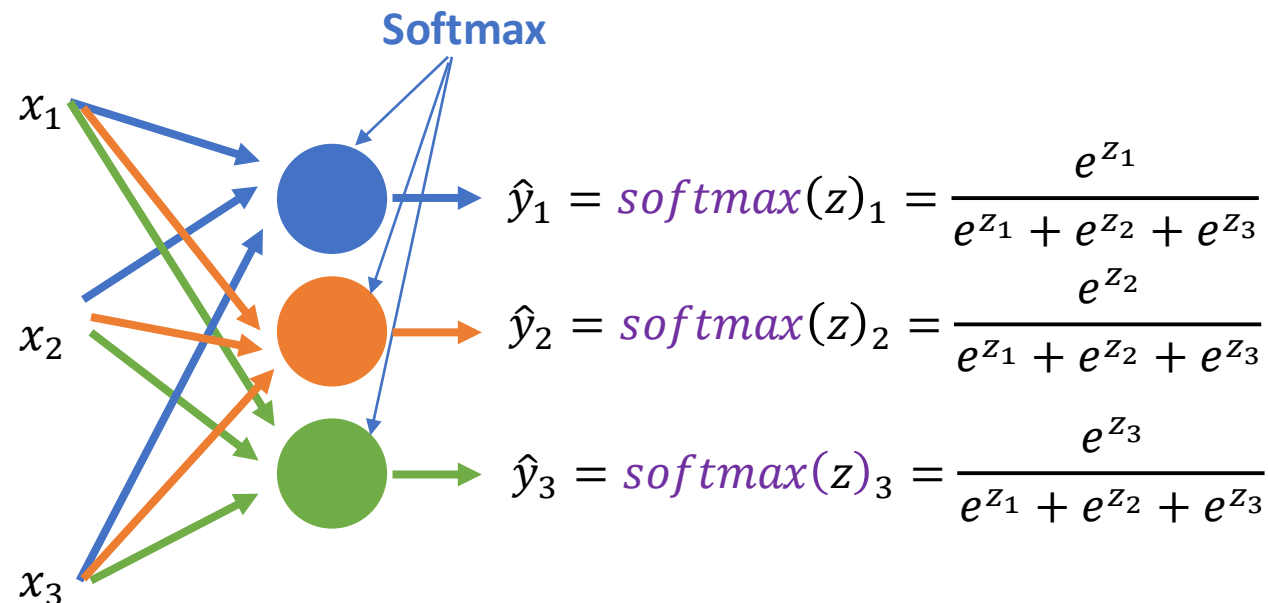
Target Label	Target Vector
Drizzle	[1, 0, 0]
Light	[0, 1, 0]
Heavy	[0, 0, 1]

Softmax Function

Softmax function computes categorical probability distribution vector from a vector of real numbers. Given a vector z , the probability of belonging to class i is:

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \in [0,1]$$

In neural networks, z_i is commonly the output of the i -th neuron model of the output layer.



Categorical Cross-Entropy Loss

Given true categorical probability vector y and predicted categorical probability vector $\hat{y}$ of K classes, categorical cross-entropy is given below.

$$CCE(y, \hat{y}) = - \sum_{i=1}^K y_i \log(\hat{y}_i)$$

Example: Let $y = [1, 0, 0]$ and $\hat{y} = [0.8, 0.1, 0.1]$,

$$CCE(y, \hat{y}) = -1 \log(0.8) - 0 \log(0.1) - 0 \log(0.1)$$

Given a dataset $D = \{(x^{(1)}, y^{(1)}), \dots, (x^{(n)}, y^{(n)})\}$ where $y^{(i)} \in \{0, 1\}$, the loss is given by:

$$J_{CCE}(\mathbf{w}) = \frac{1}{n} \sum_{i=1}^n CCE(y^{(i)}, \hat{y}^{(i)})$$

Target Label	y_1	y_2	y_3
Drizzle	1	0	0
Light	0	1	0
Heavy	0	0	1

Classification: Rain Intensity

Given data point $x \in \mathbb{R}^d$, predict rain intensity (drizzle, light, heavy), $y \in \{0,1\}^3$.

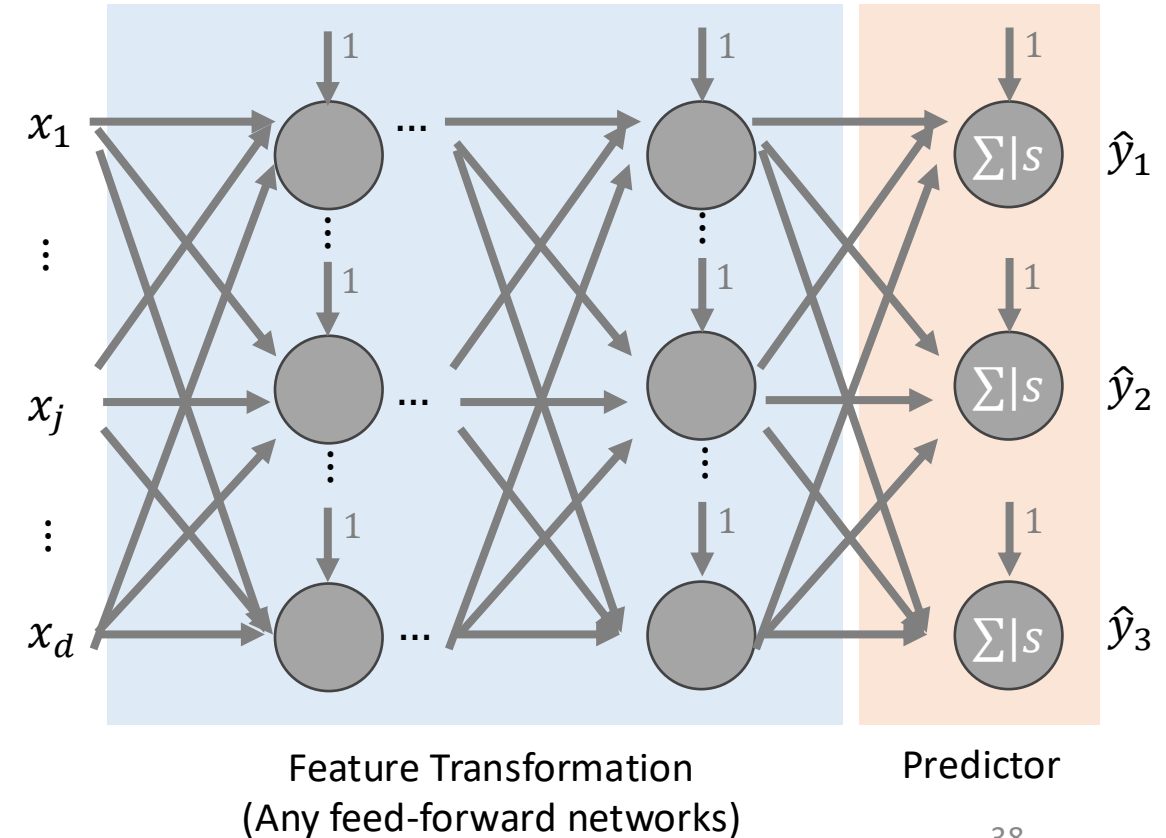
Multi-class classification problem.

Predictor:

3 neurons with Softmax activation

Loss function:

$$J_{CCE}(w) = -\frac{1}{n} \sum_{i=1}^n \sum_{j=1}^K y_j^{(i)} \log(\hat{y}_j^{(i)})$$



PollEV: Rain Intensity and Cyclone Risk

Given data point $x \in \mathbb{R}^d$, predict rain intensity (drizzle, light, heavy) and cyclone risk (low, high).

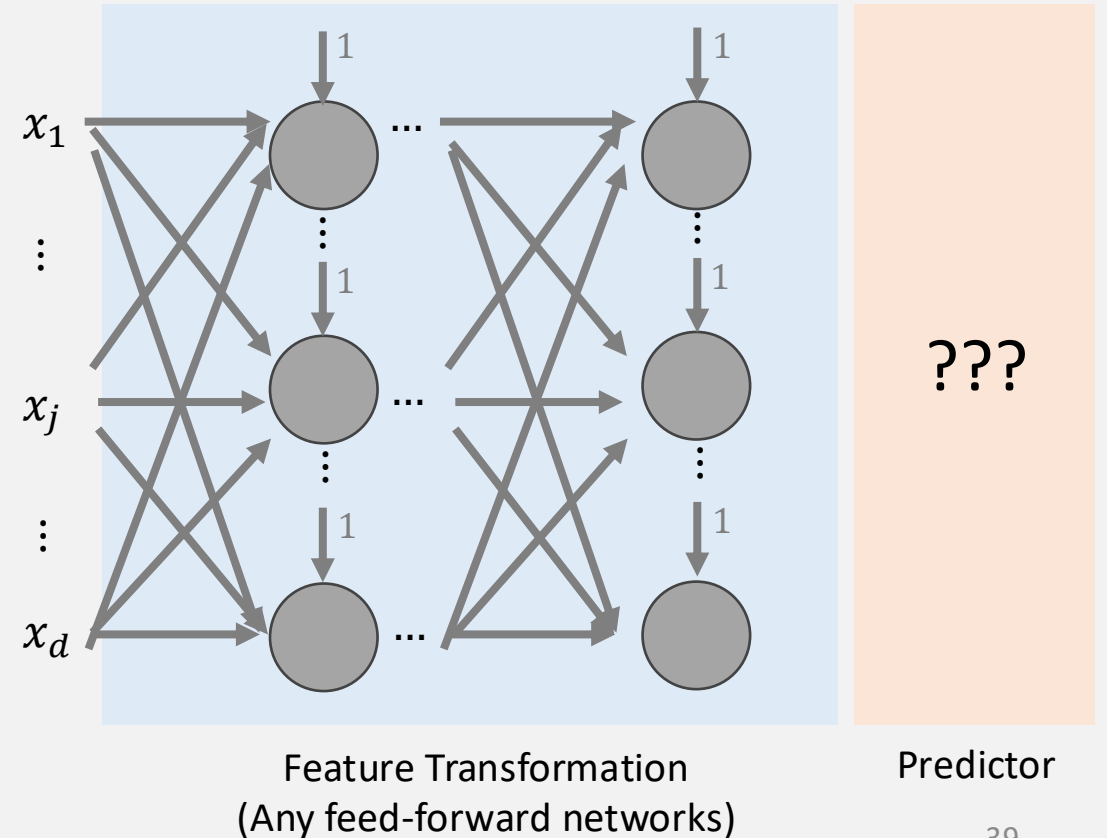
??? problem.

Predictor:

???

Loss function:

???



PolEV: Rain Intensity and Cyclone Risk

Given data point $x \in \mathbb{R}^d$, predict rain intensity (drizzle, light, heavy) and cyclone risk (low, high), i.e., $y \in \{0,1\}^4$.

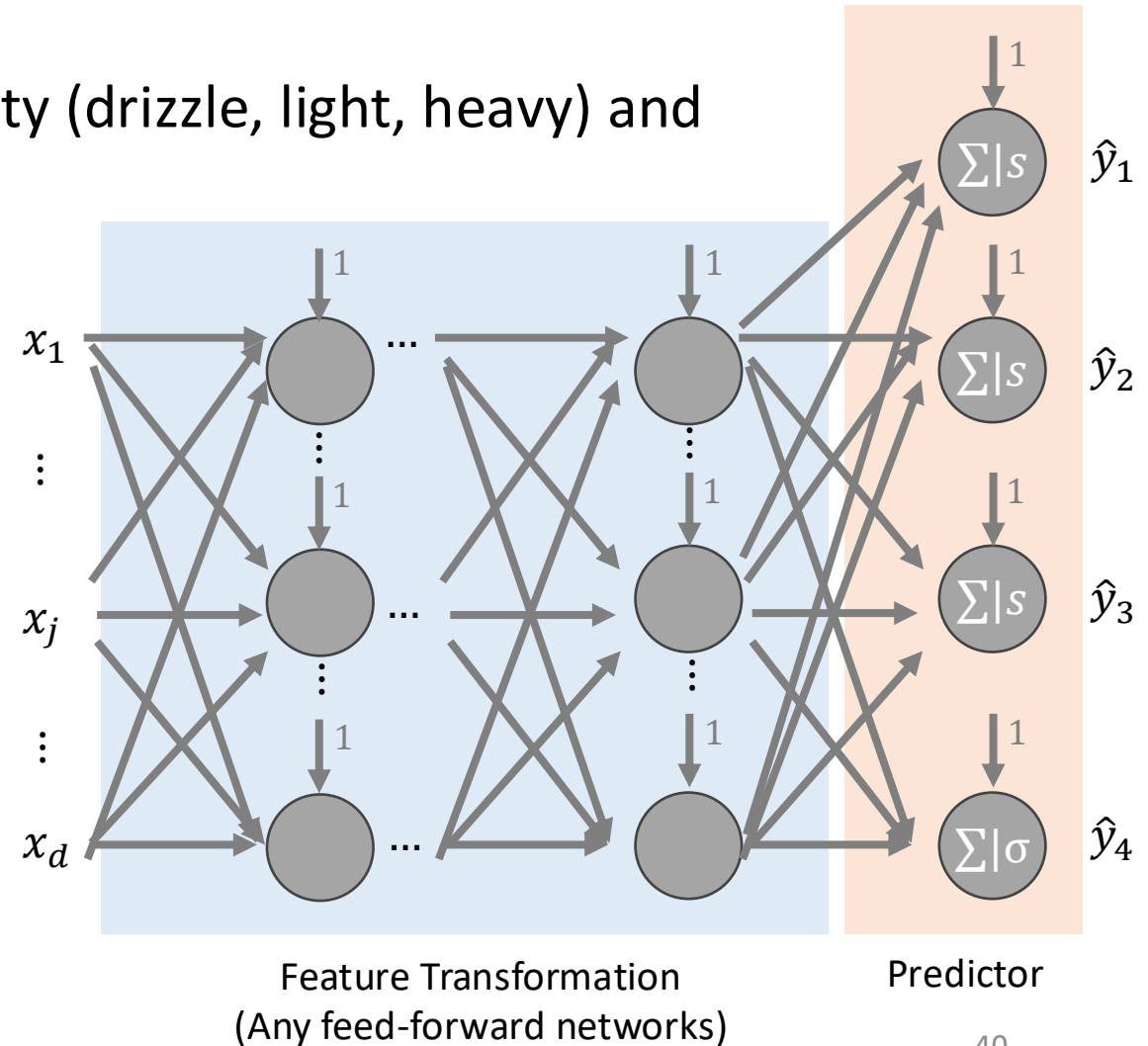
Multi-label classification problem.

Predictor:

- 3 neurons for rain intensity (Softmax)
- 1 neuron for cyclone risk (sigmoid)

Loss function:

- CCE for rain intensity +
- BCE for cyclone risk



Outline

- Neural networks
 - Neuron model
 - Artificial neural networks
 - Feed-forward neural networks
 - Fully-connected neural networks
- Tasks for feed-forward neural networks
 - Regression: single/multi output
 - Classification: binary, multi-class
- **Learning via gradient descent**
 - Backpropagation

Learning a Feedforward Neural Network

Suppose that you have a feed-forward neural network $h_{\mathbf{w}}$

The aim to solve a task involving loss function $J(\mathbf{w})$.

- $J(\mathbf{w})$ is a function of all the weights.
- We can find the optimal model by minimizing the function.

Background: Minimizing a Function

Consider a function with many parameters $\mathbf{w} = [w_1, w_2, \dots]^T$, given by $f(\mathbf{w})$.

A condition for the **minimum** of this function is when:

- Gradient $\nabla f(\mathbf{w}) = 0$.
- In other words: Partial derivative $\frac{\partial f(\mathbf{w})}{\partial w_i} = 0$ for all w_i .

Learning via Gradient Descent

Model Weights

- Step1: Start at some w (e.g., randomly initialized).
- Step2: Update w a step to the opposite direction of the gradient (i.e., towards lower loss)

Loss function

$$w_j \leftarrow w_j - \gamma \frac{\partial J(w)}{\partial w_j}.$$

Learning Rate

- Repeat Step 2 until termination criterion is satisfied.
 - E.g., change between steps is small, maximum number of steps is reached, etc

How do we compute $\frac{\partial J(w)}{\partial w_j}$ if we use neural networks?

Background: Chain Rule (1)

- Let us be given a function that is a composition of functions:

$$l = h(g(f(x)))$$

- The chain rule is a formula in calculus used to compute the derivative of a composition of functions.
- Chain rule: the derivative of l with respect to x is given by

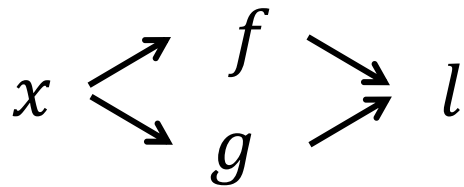
$$\frac{dl}{dx} = \frac{dh}{dg} \frac{dg}{df} \frac{df}{dx}$$

Background: Chain Rule (2)

- The chain rule can also be used to compute the derivative of l :

$$l = h(f(x), g(x))$$

- Chain rule: the derivative of l with respect to x is given by:



$$\frac{dl}{dx} = \frac{\partial h}{\partial f} \frac{df}{dx} + \frac{\partial h}{\partial g} \frac{dg}{dx}$$

Backpropagation

Backpropagation is an algorithm to efficiently compute gradients $\frac{\partial J(\mathbf{w})}{\partial \mathbf{w}_j}$ of feed-forward neural networks.

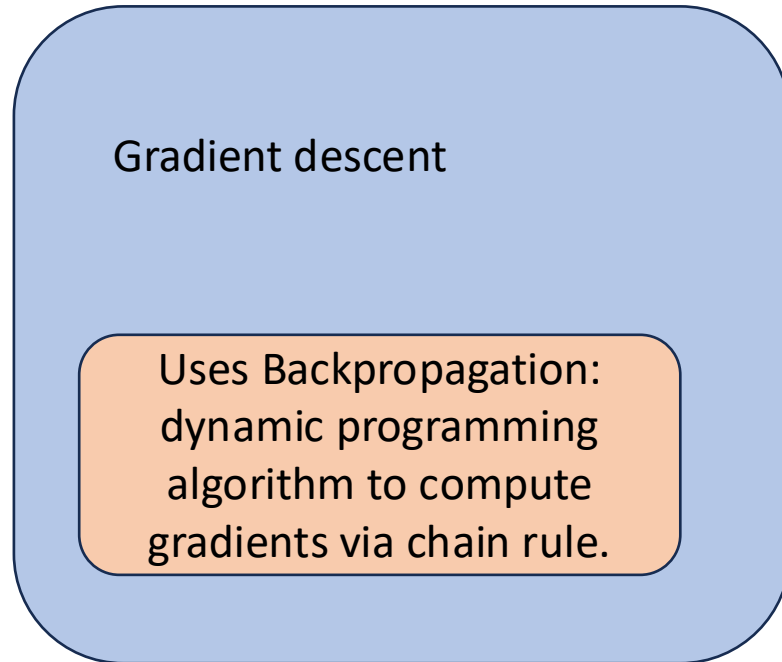
Main idea: use dynamic programming to apply chain rule consistently.

Once the gradients are computed, use them in a single iteration of (stochastic) gradient descent.

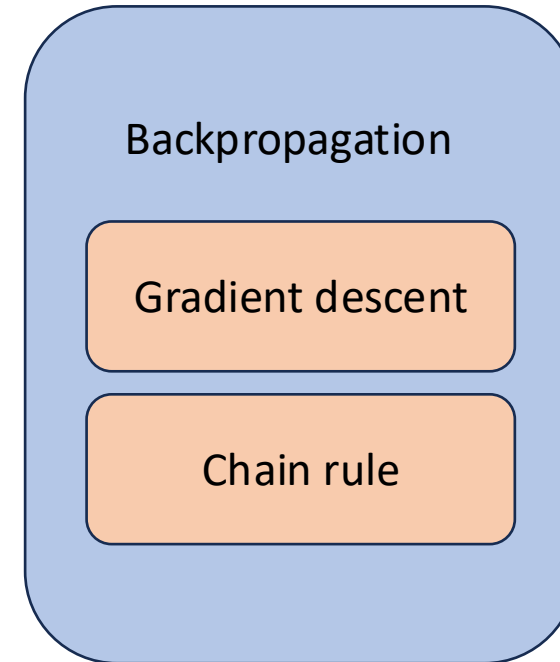
$$\mathbf{w}_j \leftarrow \mathbf{w}_j - \gamma \frac{\partial J(\mathbf{w})}{\partial \mathbf{w}_j}$$

Naming conventions: Backpropagation

Standard view (also taken in this course)



View sometimes used in colloquial/non-scientific settings.



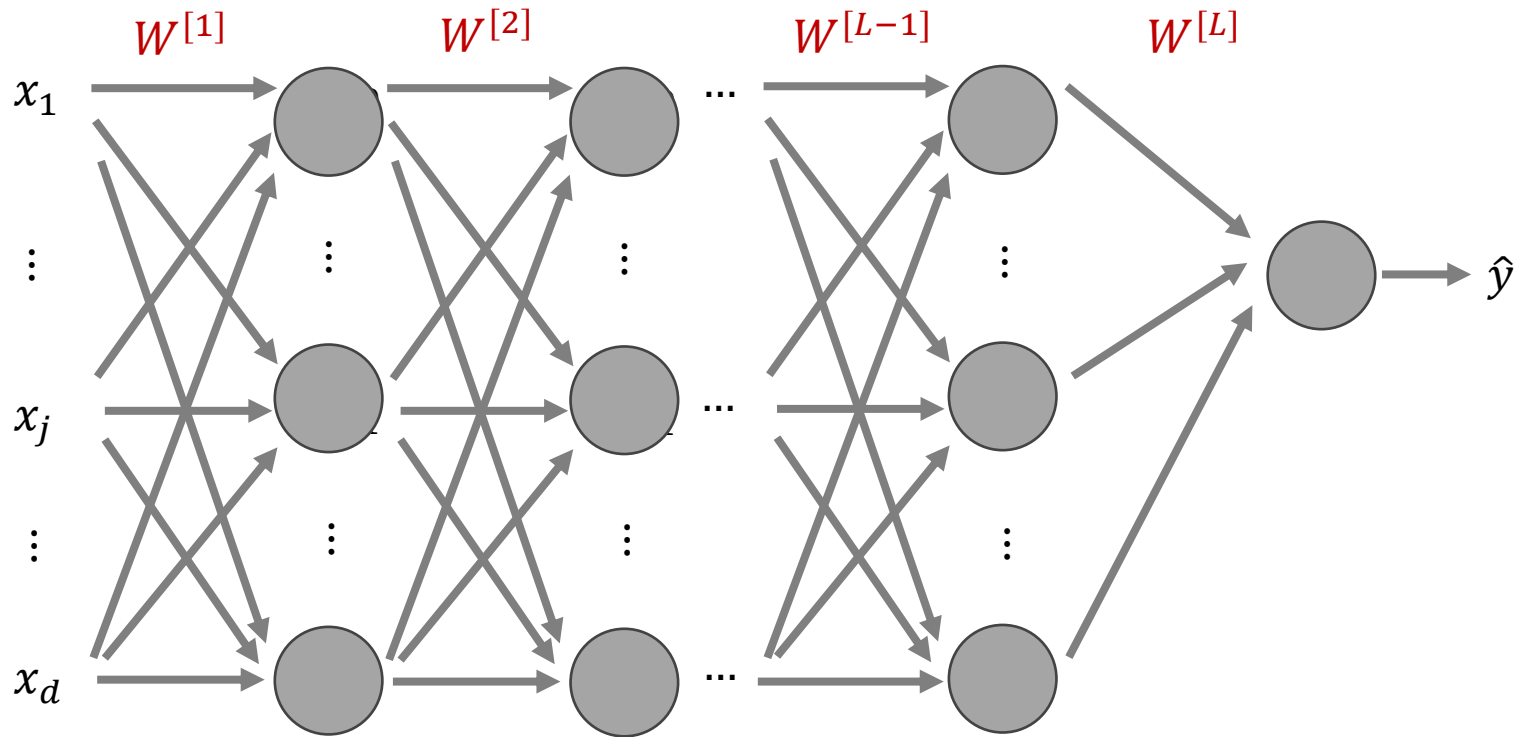
Mathematics for ML, Deisenroth,
Faisal, Ong.

For training deep neural network models, the **backpropagation algorithm** (Kelley, 1960; Bryson, 1961; Dreyfus, 1962; Rumelhart et al., 1986) is an efficient way to compute the gradient of an error function with respect to the parameters of the model.

backpropagation

Recall: FCNN and Matrix Multiplication

Multi-layer (No bias)



$$\hat{y} = W^{[L]T} g^{[L-1]} \left(W^{[L-1]T} g^{[L]} \left(W^{[L]T} \dots g^{[2]} \left(W^{[2]T} g^{[1]} \left(W^{[1]T} \mathbf{x} \right) \right) \right) \right) = \hat{y}$$

Backpropagation for FCNN (1)

Notations and assumptions:

- d_l is the dimension/width of layer l
- Dimensions: $W^{[l]} \in \mathbb{R}^{d_{l-1} \times d_l}, z^{[l]} \in \mathbb{R}^{d_l}, a^{[l]} \in \mathbb{R}^{d_l}$
- Pre-activations: $z^{[l]} = (W^{[l]})^T a^{[l-1]}, a^{[0]} = x$
- Post-activations: $a^{[l]} = g^{[l]}(z^{[l]})$
- Loss $J(w), w := \{W^{[l]}\}$ for all layers l .
 - J for short.

Backpropagation for FCNN (2)

The main object we are interested in is

$$\frac{\partial J}{\partial W^{[l]}}$$

Using chain rule, we can decompose this into:

$$\frac{\partial J}{\partial W^{[l]}} = \frac{\partial J}{\partial z^{[l]}} \frac{\partial z^{[l]}}{\partial W^{[l]}}$$

We will show that we can express $\frac{\partial J}{\partial z^{[l]}}$ as a recurrence.

Backpropagation for FCNN (3)

$$\frac{\partial J}{\partial W^{[l]}} = \frac{\partial J}{\partial z^{[l]}} \frac{\partial z^{[l]}}{\partial W^{[l]}}$$

Let $\delta^{[l]} = \frac{\partial J}{\partial z^{[l]}} \in \mathbb{R}^{d_l \times 1}$, We can express $\delta^{[l]}$ as a recurrence:

$$\delta^{[l]} = \frac{\partial J}{\partial z^{[l]}} = \frac{\partial J}{\partial a^{[l]}} \frac{\partial a^{[l]}}{\partial z^{[l]}} = (W^{[l+1]} \delta^{[l+1]}) \odot g^{[l]'}(z^{[l]})$$

$$\text{Thus, } \frac{\partial J}{\partial W^{[l]}} = \frac{\partial J}{\partial z^{[l]}} \frac{\partial z^{[l]}}{\partial W^{[l]}} = a^{[l-1]} (\delta^{[l]})^T \in \mathbb{R}^{d_{l-1} \times d_l}$$

Note: $\odot$ denotes elementwise multiplication.

Backpropagation for FCNN: Algorithm

- Perform forward pass:

- For $l = 1, \dots, L$:

- $z^{[l]} = (W^{[l]})^T a^{[l-1]}$

- $a^{[l]} = g^{[l]}(z^{[l]})$ with $a^{[0]} = x$

- Perform backward pass

- Output layer:

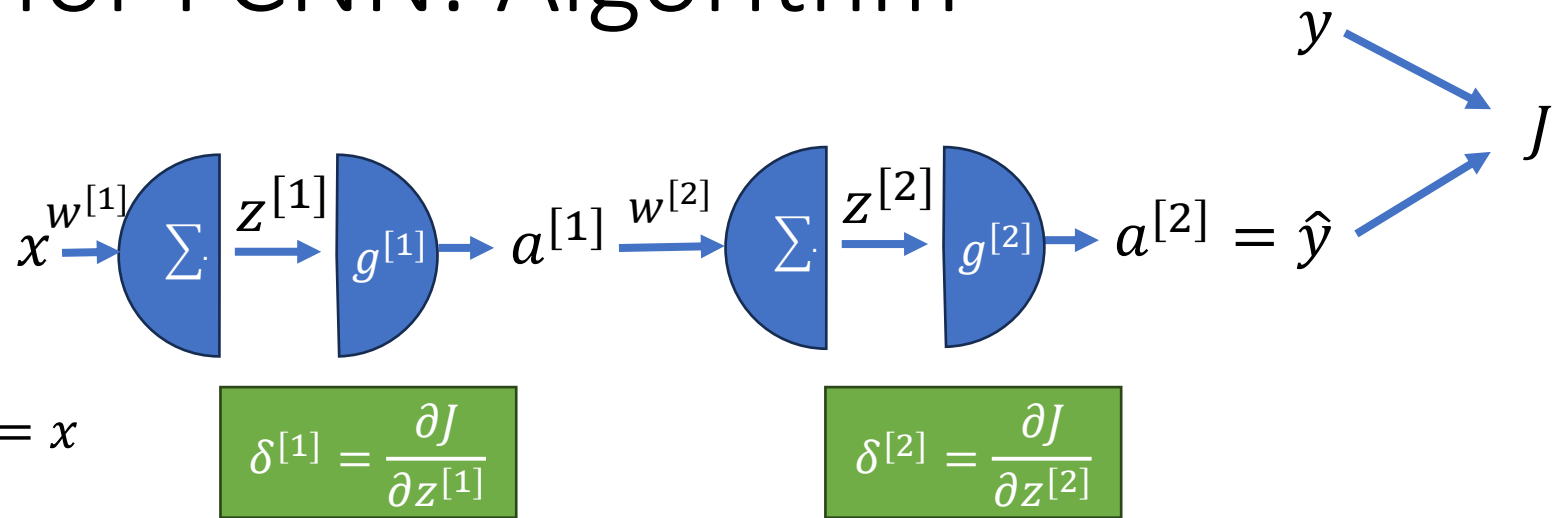
- $\delta^{[L]} = \frac{\partial J}{\partial z^{[L]}} = \frac{\partial J}{\partial a^{[L]}} \odot g^{[L]'}(z^{[L]})$

- Hidden layers, for $l = L - 1, \dots, 1$:

- $\delta^{[l]} = \frac{\partial J}{\partial z^{[l]}} = (W^{[l+1]} \delta^{[l+1]}) \odot g^{[l]'}(z^{[l]})$

- $\frac{\partial J}{\partial W^{[l]}} = a^{[l-1]} (\delta^{[l]})^T$

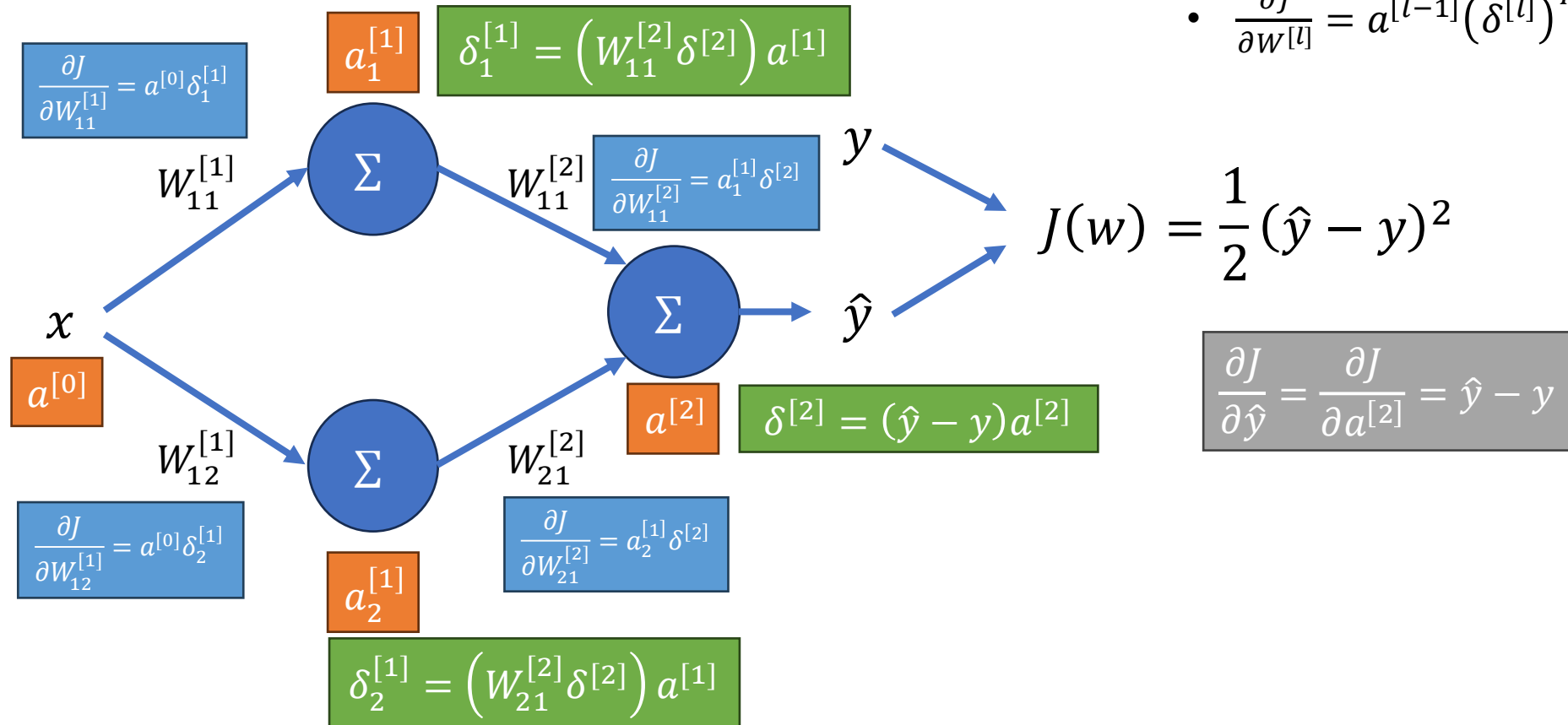
- Note: $\odot$ denotes elementwise multiplication.



Example: Backpropagation for a Simple FCNN

Let $g^{[l]}$ be identity for all layers l . Thus, $a^{[l]} = z^{[l]}$.

- $\delta^{[L]} = \frac{\partial J}{\partial a^{[L]}} \odot a^{[L]}$
- $\delta^{[l]} = (W^{[l+1]} \delta^{[l+1]}) \odot a^{[l]}$
- $\frac{\partial J}{\partial w^{[l]}} = a^{[l-1]} (\delta^{[l]})^T$



Backpropagation in Practice

- Automatic differentiation (autodiff) is a set of techniques to automatically compute the derivative of a function.
- Backpropagation is implemented as a **reverse-mode** autodiff.
- Built-in on many machine learning libraries/frameworks
- **PyTorch example:**

```
y_pred = model(data)
loss = (y_pred - y_true).mean()
loss.backward() # this
```

Summary

- Neural networks
 - Neuron model: $a = g\left(\sum_{j=0}^d w_j x_j\right)$,
 - g activation function, usually non-linear
 - Artificial neural networks:
 - A network of computing functions, usually neuron models
 - Feed-forward neural networks (FFNN): DAG neural networks
 - Fully-connected neural networks (FCNN): FCNN with neuron models
- Tasks for feed-forward neural networks
 - Regression:
 - Single: 1 output neuron, MSE loss
 - Multi output: multiple output neurons, MSE loss sum for each output component
 - Classification:
 - Binary: 1 output neuron with sigmoid activation, BCE loss
 - Multi-class: multiple output neurons with softmax activation, categorical cross-entropy (CCE) loss
- Learning via gradient descent
 - Backpropagation: apply chain rule using dynamic programming to compute gradients $\frac{\partial J(\mathbf{w})}{\partial w_j}$

Coming Up Next Week

- Other types of neural networks
 - Convolutional neural networks
 - ...
- Prof Conghui will take over!

To Do

- **Lecture Training 8**
 - +550 EXP
 - +100 Early bird bonus

